Web Programming Using Python and JavaScript Lab

[SEC035]

Lab Report Submitted to

Bachelor of Technology

In

Computer Science Engineering

SEMESTER – 4th

Submitted by

Ananya Sharma – 2401010056

SECTION - B

Course Teacher

Ms. Mansi Kajal

School of Engineering & Technology

K. R. MANGALAM UNIVERSITY

INDEX

S. No.	Experiment Title	Date	Signature
1	Personal Portfolio Website (HTML, CSS, JS)	15 .01.2026	
2	Company Landing Page – Bright Tech Solution	22.01.2026	
3	Interactive Quiz Application	30.01.2026	
4	Dynamic To-Do List (JavaScript)	12.02.2026	
5	Simple Blog Application (Flask)	2.04.2026	
6	Contact Management System (Flask)	9.04.2026	
7	E-Commerce Website (HTML, CSS, JS)	10.04.2026	

EXPERIMENT NO – 1

AIM - To design and develop a personal portfolio website using HTML, CSS, and JavaScript that displays personal information, skills, projects, services, and contact details in a structured, attractive, and interactive manner.

Technologies Used -

- **HTML – For creating the structure of the website**
- **CSS – For styling, layout, colors, and responsiveness**
- **JavaScript – For adding interactivity like buttons, show/hide, and navigation effects**
- **Web Browser – To run and test the website**

Explanation –

1️⃣ HTML Structure

- **index.html contains the main structure of the portfolio.**
- **Semantic tags such as <header>, <nav>, <section>, and <footer> are used.**
- **Separate HTML pages are created for different services like frontend, database, and Java.**

3️⃣ CSS Styling

- **style.css is linked externally to all HTML files.**
- **CSS is used for:**
  - **Layout alignment**
  - **Background colors and fonts**
  - **Spacing using margin and padding**
  - **Styling buttons, cards, and sections**

🟦 JavaScript Functionality

- **script.js is used to:**
  - **Handle button clicks**
  - **Add interactive effects**
  - **Improve navigation experience**
  - **Add show/hide functionality**

CODE-

```html
<!--
Project Title: Personal Portfolio Website
Name: Ananya Sharma
Date: 21 January 2026
-->

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Personal Portfolio</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <!-- Header / Navigation -->
    <header id="header">
        <nav class="navbar">
            <h1 class="logo">Ananya Sharma</h1>
            <ul class="nav-links" id="navLinks">
                <li><a href="#about">About</a></li>
                <li><a href="#skills">Skills</a></li>
                <li><a href="#projects">Projects</a></li>
                <li><a href="#services">Services</a></li>
                <li><a href="#contact">Contact</a></li>
            </ul>
        </nav>

        <div class="hero">
            <h2 id="welcomeText">Hello, I'm Ananya 👋</h2>
            <p>Creative Student | Web Development Enthusiast</p>
            <a href="#projects" class="btn" id="viewWorkBtn">View My Work</a>
        </div>
    </header>
```

```html
<!-- About Me Section -->
<section id="about" class="section">
    <h2>About Me</h2>
    <p>
        Hello! My name is Ananya Sharma, and I am a passionate and motivated student
        with a strong interest in web development and emerging technologies.
        I enjoy designing clean, user-friendly websites and turning ideas into
        functional digital solutions. I have hands-on experience in building
        projects using HTML and CSS, and I am continuously improving my skills
        in modern web technologies.
    </p>
</section>

<!-- Skills Section -->
<section id="skills" class="section">
    <h2>Skills</h2>
    <ul class="skills-list">
        <li>HTML5</li>
        <li>CSS3</li>
        <li>Basic JavaScript</li>
        <li>SQL & DBMS Basics</li>
        <li>Problem Solving</li>
        <li>Communication Skills</li>
    </ul>
</section>

<!-- Projects Section -->
<section id="projects" class="section">
    <h2>Projects</h2>

    <div class="project-card">
        <h3>Instock – Inventory Management Website</h3>
        <p>
            A responsive inventory management website developed using HTML and CSS.
            This project helps in organizing and displaying stock information in a
            clean and structured layout.
        </p>
    </div>

    <div class="project-card">
        <h3>Recipe Haven – Recipe Sharing Website</h3>
        <p>
            A visually appealing recipe website built using HTML and CSS where users
            can explore different recipes with attractive layouts and easy navigation.
        </p>
    </div>
</section>

<!-- Services Section -->
<section id="services" class="section light-bg">
    <h2>Services</h2>
    <div class="services-grid">
        <div class="service-card" onclick="highlightService(this)">
```

```
            I create responsive and visually appealing web pages using HTML, CSS,
            and basic JavaScript. The focus is on clean design, proper layout,
            and a user-friendly experience that works on all devices.
        </p>
        <a href="frontend.html" class="service-btn" target="_blank">Learn More</a>
    </div>

    <div class="service-card" onclick="highlightService(this)">
        <h3>Database Management</h3>
        <p>
            I work with SQL and DBMS to design, manage, and query databases efficiently.
            This includes creating tables, managing records, and ensuring data integrity
            for small to medium-scale applications.
        </p>
        <a href="database.html" class="service-btn" target="_blank">Learn More</a>
    </div>

    <div class="service-card" onclick="highlightService(this)">
        <h3>Java Programming</h3>
        <p>
            I have experience in writing Java programs for problem-solving,
            building basic applications, and implementing object-oriented concepts.
            I can develop functional and structured code for academic projects.
        </p>
        <a href="java.html" class="service-btn" target="_blank">Learn More</a>
    </div>
</div>
ection>


- Contact Section -->
ction id="contact" class="section contact-section">
>Contact Me</h2>
class="contact-text">


rm id="contactForm" class="contact-form">
<div class="form-group">
    <input type="text" id="name" placeholder="Your Full Name" required>
</div>

<div class="form-group">
    <input type="email" id="email" placeholder="Your Email Address" required>
</div>

<div class="form-group">
    <textarea id="message" placeholder="Your Message" rows="5" required></textarea>
</div>

<button type="submit" class="btn">Send Message</button>
```

CSS CODE

```
/* General Styles */
body {
    margin: 0;
    font-family: Arial, Helvetica, sans-serif;
    background-color: #f4f6f8;
    color: #333;
    line-height: 1.6;
}

/* Header & Navigation */
#header {
    background: linear-gradient(to right, #0f2027, #203a43, #2c5364);
    color: white;
    padding-bottom: 40px;
}

.navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 20px 40px;
}

.logo {
    font-size: 28px;
    font-weight: bold;
}

.nav-links {
    list-style: none;
    display: flex;
    gap: 20px;
}

.nav-links li a {
    text-decoration: none;
    color: white;
    font-size: 16px;
}

.nav-links li a:hover {
    color: #ffcc00;
}

/* Hero Section */
.hero {
    text-align: center;
    margin-top: 40px;
}

.hero h2 {
    font-size: 36px;
    margin-bottom: 10px;
}
```

```
.hero p {
    font-size: 18px;
    margin-bottom: 20px;
}

.hero button {
    padding: 8px 18px;
    font-size: 14px;
    background-color: #ffcc00;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    font-weight: bold;
}


.btn:hover {
    background-color: #e6b800;
}

/* Common Section Styling */
.section {
    padding: 60px 80px;
    background-color: white;
    margin: 30px auto;
    max-width: 1000px;
    border-radius: 10px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}

.section h2 {
    text-align: center;
    margin-bottom: 30px;
    color: #2c5364;
}

.light-bg {
    background-color: #f9fafb;
}

/* About Section */
#about p {
    max-width: 750px;
    margin: auto;
    text-align: center;
    font-size: 17px;
}

/* Skills Section */
.skills-list {
    list-style: none;
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
    gap: 15px;
    padding: 0;
```

```
/* Projects Section */
.project-card {
    background-color: #f9f9f9;
    border-left: 5px solid #2c5364;
    padding: 20px;
    margin-bottom: 20px;
    border-radius: 6px;
}

.project-card h3 {
    margin-top: 0;
    color: #2c5364;
}

/* Services Section */
.services-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 20px;
}

.service-card {
    background-color: #f9f9f9;
    padding: 20px;
    border-radius: 6px;
    text-align: left;
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);
}

.service-card h3 {
    margin-bottom: 12px;
    color: #2c5364;
}

.service-card p {
    font-size: 15px;
    line-height: 1.5;
    margin-bottom: 15px;
}

.service-btn {
    display: inline-block;
    padding: 8px 16px;
    background-color: #2563eb;
    color: white;
    border-radius: 5px;
    text-decoration: none;
    font-weight: bold;
}

.service-btn:hover {
    background-color: #1d4ed8;
}
```

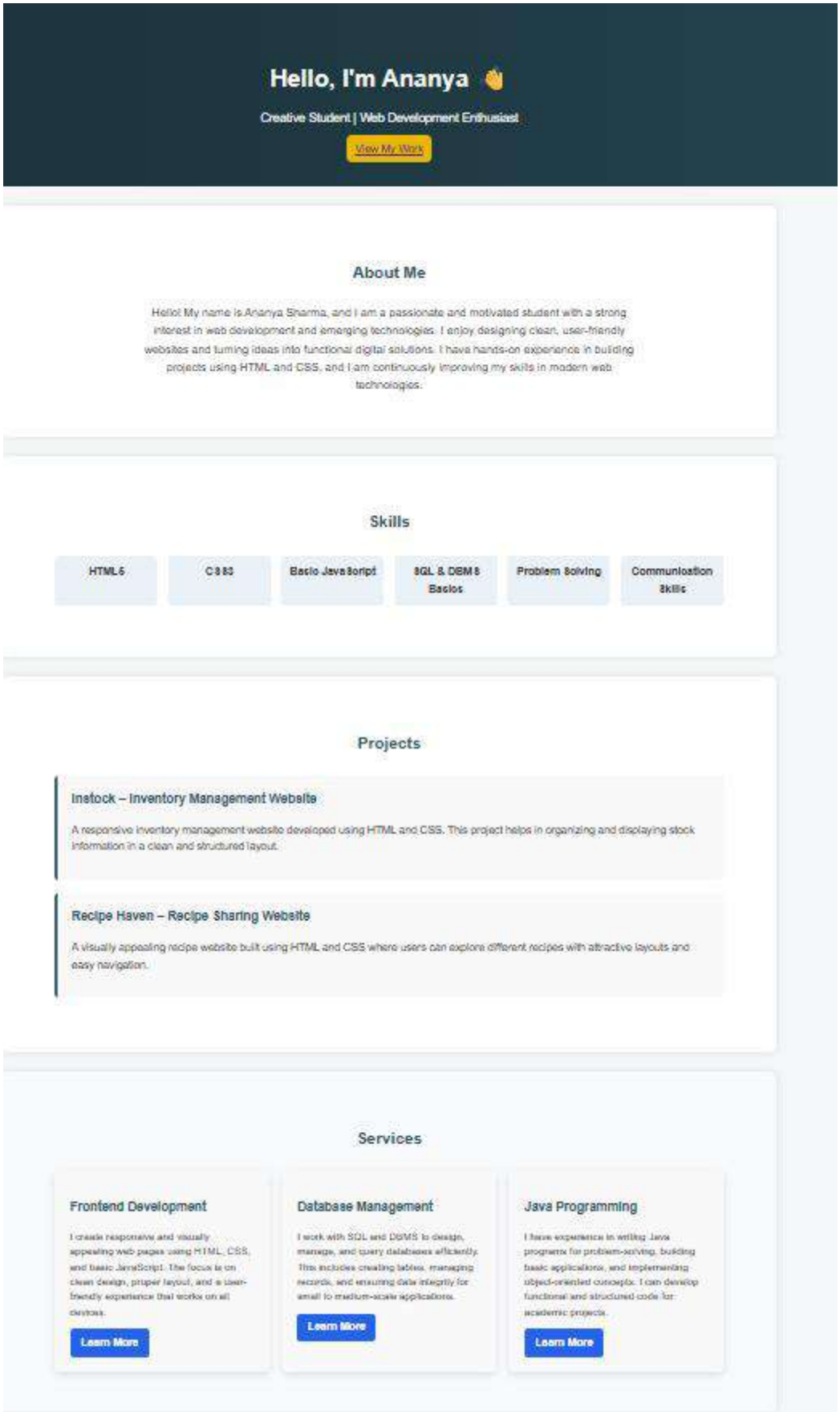
```
#contact p {
    text-align: center;
    font-size: 16px;
    margin: 8px 0;
}

/* Footer */
.footer {
    background-color: #0f2027;
    color: white;
    text-align: center;
    padding: 15px 0;
    margin-top: 40px;
}
.active-service {
    border: 2px solid #4f46e5;
    transform: scale(1.03);
    transition: 0.3s ease;
}
.service-card {
    border: 2px solid transparent;
    transition: 0.3s ease;
    cursor: pointer;
}

.service-card:hover {
    border: 2px solid #4f46e5;
}

.active-service {
    border: 2px solid #4f46e5 !important;
    box-shadow: 0 0 10px rgba(79, 70, 229, 0.3);
    transform: scale(1.03);
}
.contact-section {
    text-align: center;
    padding: 60px 20px;
}

.contact-text {
    max-width: 450px;
    margin: 0 auto 30px;
    text-align: center;
    line-height: 1.6;
}


.contact-form {
    max-width: 450px;
    margin: 0 auto;
    background: #f9fafb;
    padding: 30px;
    border-radius: 12px;
    box-shadow: 0 8px 20px rgba(0,0,0,0.05);
}
```

```css
.form-group {
    margin-bottom: 18px;
}

.contact-form input,
.contact-form textarea {
    width: 100%;
    padding: 12px 14px;
    border-radius: 8px;
    border: 1px solid #ccc;
    font-size: 15px;
    outline: none;
}

.contact-form input:focus,
.contact-form textarea:focus {
    border-color: #4f46e5;
    box-shadow: 0 0 0 2px rgba(79,70,229,0.1);
}

.btn {
    width: 100%;
    padding: 12px;
    font-size: 16px;
    border-radius: 8px;
}

.form-message {
    margin-top: 15px;
    font-weight: 600;
}
```

JavaScript Code-

```javascript
document.getElementById("viewWorkBtn").addEventListener("click", function (e) {
    e.preventDefault();
    document.getElementById("projects").scrollIntoView({ behavior: "smooth" });
});


function highlightService(card) {
    const cards = document.querySelectorAll(".service-card");
    cards.forEach(c => c.classList.remove("active-service"));
    card.classList.add("active-service");
}
```

OUTPUT


Hello, I'm Ananya 👋
Creative Student | Web Development Enthusiast
View My Work
About Me
Hello! My name is Ananya Sharma, and I am a passionate and motivated student with a strong interest in web development and emerging technologies. I enjoy designing clean, user-friendly websites and turning ideas into functional digital solutions. I have hands-on experience in building projects using HTML and CSS, and I am continuously improving my skills in modern web technologies.
Skills
HTML 5
CSS3
Basic JavaScript
SQL & DBMS Basics
Problem Solving
Communication Skills
Projects
Instock – Inventory Management Website
A responsive inventory management website developed using HTML and CSS. This project helps in organizing and displaying stock information in a clean and structured layout.
Recipe Haven – Recipe Sharing Website
A visually appealing recipe website built using HTML and CSS where users can explore different recipes with attractive layouts and easy navigation.
Services
Frontend Development
I create responsive and visually appealing web pages using HTML, CSS, and basic JavaScript. The focus is on clean design, proper layout, and a user-friendly experience that works on all devices.
Learn More
Database Management
I work with SQL and DBMS to design, manage, and query databases efficiently. This includes creating tables, managing records, and ensuring data integrity for small to medium-scale applications.
Learn More
Java Programming
I have experience in writing Java programs for problem-solving, building basic applications, and implementing object-oriented concepts. I can develop functional and structured code for academic projects.
Learn More

Services

Frontend Development

I create responsive and visually appealing web pages using HTML, CSS, and basic JavaScript. The focus is on clean design, proper layout, and a user-friendly experience that works on all devices.

Learn More

Database Management

I work with SQL and DBMS to design, manage, and query databases efficiently. This includes creating tables, managing records, and ensuring data integrity for small to medium-scale applications.

Learn More

Java Programming

I have experience in writing Java programs for problem-solving, building basic applications, and implementing object-oriented concepts. I can develop functional and structured code for academic projects.

Learn More

Ananya Sharma

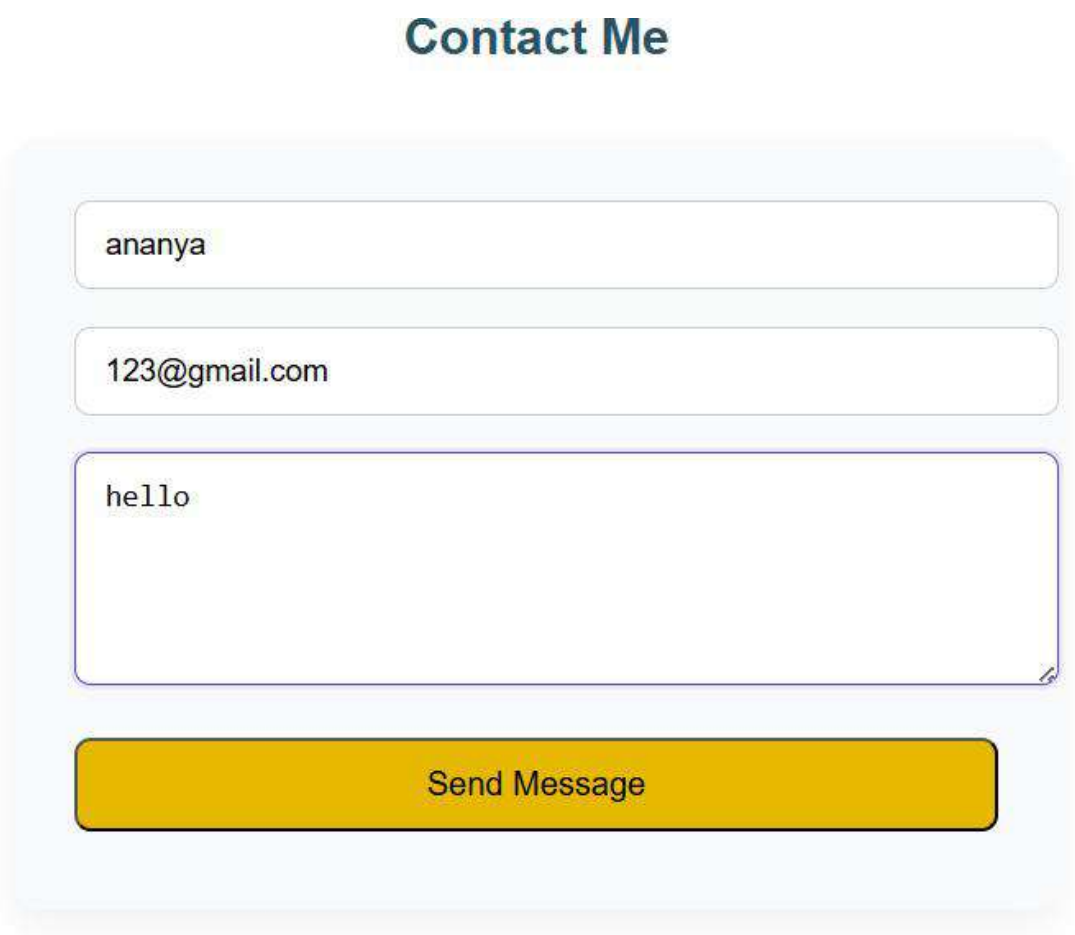
About Skills Projects Services Contact

Frontend Development

I specialize in building responsive and visually appealing web pages using HTML, CSS, and basic JavaScript. The focus is on creating intuitive layouts, consistent styling, and smooth user interactions.

I ensure websites are mobile-friendly, load quickly, and follow modern web standards. This service is ideal for portfolio websites, landing pages, and small business websites.

© 2026 Ananya Sharma. All Rights Reserved



Result:

A fully functional, professional-looking personal portfolio website was successfully created using HTML, CSS, and JavaScript.

EXPERIMENT NO – 2

AIM- To design and develop a professional company landing page named Bright Solution using HTML and CSS, displaying company information, services, testimonials, and contact details in a well-structured and visually appealing manner.

Technologies Used-

- HTML – To create the structure of the landing page

- CSS – To design layout, colors, fonts, and overall appearance

- Web Browser – To test and run the website

Explanation-

🟦 HTML Structure

- **index.html contains the main structure of the landing page.**
- **Semantic tags like <header>, <section>, and <footer> are used.**
- **Sections included:**
  - **Header / Hero Section**
  - **About / Introduction**
  - **Services**
  - **Testimonials**
  - **Contact Information**
- **Separate HTML pages are created for each service (Web Development, Database, IT Consulting).**

CSS Styling

- **style.css is linked externally to all pages.**
- **CSS is used to:**
  - **Style layout and spacing**
  - **Apply background colors and fonts**
  - **Align sections and create service cards**
  - **Improve readability and visual appeal**

CODE

HTML

```html
<!--
Project Title: Company Landing Page
Name: Ananya Sharma
Date: 2026
-->

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>BrightTech Solutions</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <!-- Header / Hero Section -->
    <header class="hero">
        <div class="hero-content">
            <h1>BrightTech Solutions</h1>
            <p>Innovating Your Digital Future</p>
            <a href="#services" class="btn">Explore Our Services</a>
        </div>
    </header>

    <!-- About / Introduction Section -->
    <section id="about" class="section">
        <h2>About Us</h2>
        <p>
            BrightTech Solutions is a forward-thinking technology company dedicated to
            delivering high-quality digital solutions. We specialize in building modern,
            secure, and user-friendly applications that help businesses grow and succeed
            in the digital world.
        </p>
    </section>
```

```html
<body>

    <!-- Services Section -->
    <section id="services" class="section light-bg">
    <h2>Our Services</h2>
    <div class="services-grid">

        <a href="web-development.html" class="service-card-link">
            <div class="service-card">
                <h3>Web Development</h3>
                <p>
                    We design and develop responsive, fast, and visually appealing websites
                    tailored to meet your business needs and enhance user experience.
                </p>
            </div>
        </a>

        <a href="database-management.html" class="service-card-link">
            <div class="service-card">
                <h3>Database Management</h3>
                <p>
                    Our team provides efficient database design, management, and optimization
                    services to ensure secure and reliable data storage solutions.
                </p>
            </div>
        </a>

        <a href="it-consulting.html" class="service-card-link">
            <div class="service-card">
                <h3>IT Consulting</h3>
                <p>
                    We offer expert IT consulting services to help businesses adopt the right
                    technologies and improve their overall operational efficiency.
                </p>
            </div>
        </a>

    </div>
</section>


    <!-- Testimonials Section -->
    <section id="testimonials" class="section">
        <h2>What Our Clients Say</h2>
        <div class="testimonials-container">

            <div class="testimonial-card">
                <p>
                    "BrightTech Solutions transformed our online presence. Their team is
                    professional, responsive, and highly skilled. We are extremely satisfied
                    with their work!"
                </p>
                <h4>- Rahul Verma</h4>
            </div>
```

```html
<p>
    "The quality of service and attention to detail provided by BrightTech
    Solutions is outstanding. They delivered our project on time and exceeded
    our expectations."
</p>
<h4>- Neha Sharma</h4>
div>
```

```html
<!-- Contact Section -->
<section id="contact" class="section light-bg">
    <h2>Contact Us</h2>

    <div class="contact-info">
        <p><strong>Address:</strong> 123 Tech Street, New Delhi, India</p>
        <p><strong>Email:</strong> contact@brighttechsolutions.com</p>
        <p><strong>Phone:</strong> +91 98765 43210</p>
    </div>

    <form class="contact-form">
        <input type="text" placeholder="Your Name" required>
        <input type="email" placeholder="Your Email" required>
        <textarea rows="5" placeholder="Your Message" required></textarea>
        <button type="submit" class="btn">Send Message</button>
    </form>
</section>

<!-- Footer -->
<footer class="footer">
    <p>Designed by Ananya Sharma</p>
</footer>

/body>
/html>
```

CSS CODE

yle.css > ...

```css
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: "Segoe UI", Tahoma, sans-serif;
}

body {
    background-color: #f4f7fb;
    color: #333;
    line-height: 1.6;
}

/* Hero Section */
.hero {
    background: linear-gradient(135deg, #0f2d3a, #2563eb);
    color: white;
    padding: 110px 20px;
    text-align: center;
}

.hero h1 {
    font-size: 50px;
    margin-bottom: 12px;
    font-weight: 700;
}

.hero p {
    font-size: 20px;
    margin-bottom: 28px;
    opacity: 0.9;
}

.btn {
    display: inline-block;
    padding: 12px 26px;
    background-color: #ffcc00;
    color: #000;
    text-decoration: none;
    border-radius: 30px;
    font-weight: 600;
    transition: 0.3s ease;
}

.btn:hover {
    background-color: #ffd633;
    transform: translateY(-2px);
}

/* Sections */
.section {
    padding: 80px 20px;
    max-width: 1150px;
    margin: auto;
    text-align: center;
```

```
    position: relative;
}

.section h2::after {
    content: "";
    width: 60px;
    height: 3px;
    background: #2563eb;
    display: block;
    margin: 10px auto 0;
    border-radius: 2px;
}

.light-bg {
    background-color: #eef3fb;
}

/* Services Section */
.services-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
    gap: 25px;
}

.service-card {
    background: white;
    padding: 28px;
    border-radius: 14px;
    box-shadow: 0 10px 25px rgba(0,0,0,0.08);
    text-align: left;
    transition: 0.3s ease;
}

.service-card:hover {
    transform: translateY(-5px);
    box-shadow: 0 14px 35px rgba(0,0,0,0.12);
}

.service-card h3 {
    margin-bottom: 12px;
    color: #0f2d3a;
    font-size: 20px;
}

/* Testimonials Section */
.testimonials-container {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
    gap: 25px;
}
```

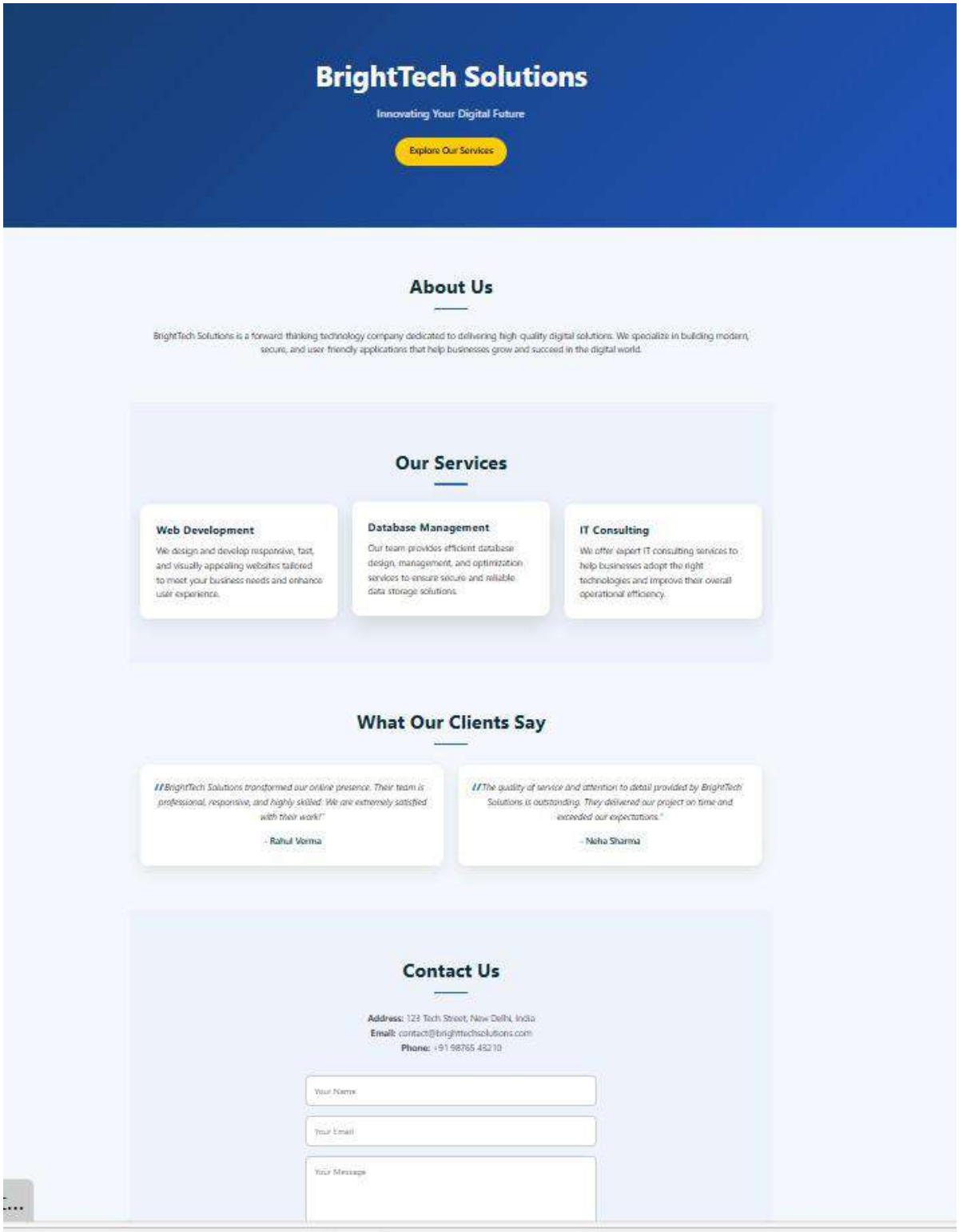
```
.testimonial-card {
    background: white;
    padding: 30px;
    border-radius: 14px;
    box-shadow: 0 10px 25px rgba(0,0,0,0.08);
    font-style: italic;
    position: relative;
}

.testimonial-card::before {
    content: "“";
    font-size: 50px;
    color: #2563eb;
    position: absolute;
    top: 10px;
    left: 20px;
}

.testimonial-card h4 {
    margin-top: 18px;
    color: #0f2d3a;
    font-style: normal;
    font-weight: 600;
}

/* Contact Section */
.contact-info {
    margin-bottom: 35px;
    font-size: 16px;
    color: #555;
}

.contact-form {
    max-width: 520px;
    margin: auto;
    display: flex;
    flex-direction: column;
    gap: 18px;
}

.contact-form input,
.contact-form textarea {
    padding: 14px;
    border-radius: 10px;
    border: 1px solid #ccc;
    font-size: 15px;
    transition: 0.2s ease;
```

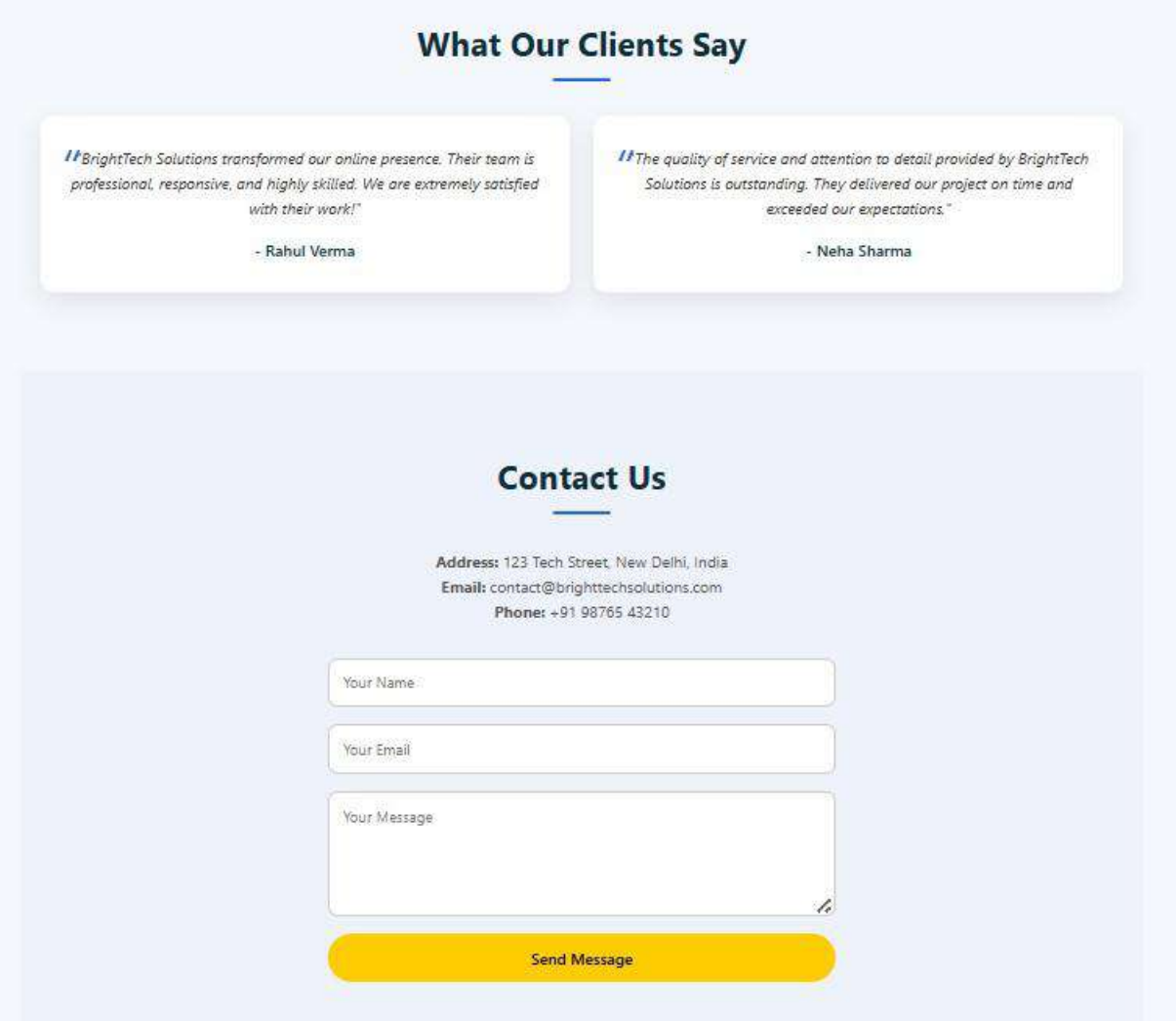
```css
.contact-form input:focus,
.contact-form textarea:focus {
    border-color: #2563eb;
    outline: none;
    box-shadow: 0 0 0 3px rgba(37,99,235,0.1);
}

.contact-form button {
    padding: 14px;
    font-size: 16px;
    border: none;
    border-radius: 30px;
    background-color: #ffcc00;
    font-weight: 600;
    cursor: pointer;
    transition: 0.3s ease;
}

.contact-form button:hover {
    background-color: #ffd633;
    transform: translateY(-2px);
}

/* Footer */
.footer {
    background: #0f2d3a;
    color: white;
    text-align: center;
    padding: 18px;
    margin-top: 50px;
    font-size: 15px;
}
.service-card-link {
    text-decoration: none;
    color: inherit;
    display: block;
}
```

💥 Features Implemented:

- **Company name and tagline displayed in hero section**
- **Services displayed in card layout with clickable links**
- **Testimonials section with customer feedback**
- **Contact section with address, email, phone, and form**
- **Clean and professional UI**

OUTPUT


BrightTech Solutions
Innovating Your Digital Future
Explore Our Services
About Us
BrightTech Solutions is a forward thinking technology company dedicated to delivering high quality digital solutions. We specialize in building modern, secure, and user friendly applications that help businesses grow and succeed in the digital world.
Our Services
Web Development
We design and develop responsive, fast, and visually appealing websites tailored to meet your business needs and enhance user experience.
Database Management
Our team provides efficient database design, management, and optimization services to ensure secure and reliable data storage solutions.
IT Consulting
We offer expert IT consulting services to help businesses adopt the right technologies and improve their overall operational efficiency.
What Our Clients Say
"BrightTech Solutions transformed our online presence. Their team is professional, responsive, and highly skilled. We are extremely satisfied with their work!"
- Rahul Verma
"The quality of service and attention to detail provided by BrightTech Solutions is outstanding. They delivered our project on time and exceeded our expectations."
- Neha Sharma
Contact Us
Address: 123 Tech Street, New Delhi, India
Email: contact@brighttechsolutions.com
Phone: +91 98765 43210
Your Name
Your Email
Your Message



Result: A complete and visually appealing company landing page for Bright Solution was successfully developed using HTML and CSS.

EXPERIMENT NO – 3

AIM- To design and develop an interactive quiz application using HTML, CSS, and JavaScript that displays questions, records user responses, tracks the score, shows results, and allows the quiz to be restarted.

Technologies Used:

- **HTML – For creating the structure of the quiz interface**
- **CSS – For styling the quiz layout and improving user experience**
- **JavaScript – For quiz logic, interactivity, timer, and result calculation**
- **Web Browser – To run and test the application**

Explanation

1️⃣ HTML Structure

HTML is used to create the layout of the quiz using semantic elements such as <header>, <section>, and <button>.
It displays:

- **Quiz title and introduction**
- **Question text**
- **Multiple-choice options (radio buttons)**
- **Next button**
- **Result section and feedback form**

CSS Styling

CSS is applied to:

- **Improve readability of questions and options**
- **Align sections and buttons properly**
- **Use colors, spacing, and borders for better user experience**
- **Make the quiz interface clean and professional**

JavaScript Functionality

JavaScript is used to:

- **Store questions and answers in an array of objects**
- **Capture user-selected answers**
- **Move between questions using button events**
- **Track and update the score**
- **Display final results and feedback messages**
- **Implement a countdown timer for each question**
- **Allow the quiz to restart**

CODE

HTML CODE

script.js index.html

index.html > html > body

```html
<!--
Project Title: Interactive Quiz Application
Name: Ananya Sharma
Date: 21 January 2026
-->

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Interactive Quiz</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <div class="quiz-container">

        <!-- Introduction -->
        <header class="quiz-header">
            <h1>Interactive Quiz</h1>
            <p>Test your knowledge with this fun and interactive quiz!</p>
        </header>

        <!-- Quiz Box -->
        <section id="quiz-box">
            <h2 id="question">Question Text</h2>

            <div class="options">
                <label>
                    <input type="radio" name="option" value="0">
                    <span id="option0">Option 1</span>
                </label>

                <label>
                    <input type="radio" name="option" value="1">
                    <span id="option1">Option 2</span>
                </label>
```

```html
            <span id="option2">Option 3</span>
        </label>

        <label>
            <input type="radio" name="option" value="3">
            <span id="option3">Option 4</span>
        </label>
    </div>

    <button id="next-btn">Next</button>
</section>

<!-- Result Box -->
<section id="result-box" class="hidden">
    <h2>Your Result</h2>
    <p id="score-text"></p>
    <p id="message-text"></p>

    <!-- Feedback Form -->
    <div class="feedback-box">
        <h3>Give Your Feedback</h3>
        <textarea id="feedback-text" placeholder="Write your feedback here..." rows="3"></textarea>
        <button id="submit-feedback-btn">Submit Feedback</button>
        <p id="feedback-msg" class="hidden"></p>
    </div>

    <button id="restart-btn">Restart Quiz</button>
</section>
```

CSS CODE

```css
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: Arial, sans-serif;
}

body {
    background: linear-gradient(to right, #0f2027, #203a43, #2c5364);
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
    color: #333;
}
```

```css
.quiz-container {
    background-color: #ffffff;
    padding: 30px 25px;
    border-radius: 12px;
    width: 90%;
    max-width: 500px;
    box-shadow: 0 10px 25px rgba(0,0,0,0.15);
}

.quiz-header {
    text-align: center;
    margin-bottom: 20px;
}

.quiz-header h1 {
    color: #203a43;
    margin-bottom: 8px;
}

.quiz-header p {
    font-size: 14px;
    color: #555;
}

#question {
    font-size: 20px;
    margin-bottom: 15px;
    color: #203a43;
}

.options label {
    display: block;
    background: #f4f6f8;
    padding: 10px 12px;
    border-radius: 6px;
    margin-bottom: 10px;
```

```css
.options label:hover {
    background: #e8f0f3;
    border-color: #203a43;
}

.options input {
    margin-right: 8px;
}

#next-btn,
#restart-btn {
    display: block;
    width: 100%;
    margin-top: 15px;
    padding: 10px;
    background-color: #ffcc00;
    border: none;
    border-radius: 6px;
    font-size: 15px;
    font-weight: bold;
    cursor: pointer;
    transition: background 0.3s;
}

#next-btn:hover,
#restart-btn:hover {
    background-color: #e6b800;
}

#result-box {
    text-align: center;
}

#result-box h2 {
    color: #203a43;
```

```
#message-text {
    font-size: 16px;
    font-weight: bold;
    color: #444;
}

.hidden {
    display: none;
}
/* ... your existing CSS remains same ... */

.feedback-box {
    margin-top: 20px;
    padding: 15px;
    border: 1px solid #ddd;
    border-radius: 8px;
    background-color: #f9fafb;
}

.feedback-box h3 {
    margin-bottom: 10px;
    color: #203a43;
}

#feedback-text {
    width: 100%;
    padding: 8px;
    border-radius: 5px;
    border: 1px solid #ccc;
    margin-bottom: 10px;
}

#submit-feedback-btn {
    padding: 8px 16px;
    background-color: #203a43;
    color: white;
    border: none;
```

JavaScript Code

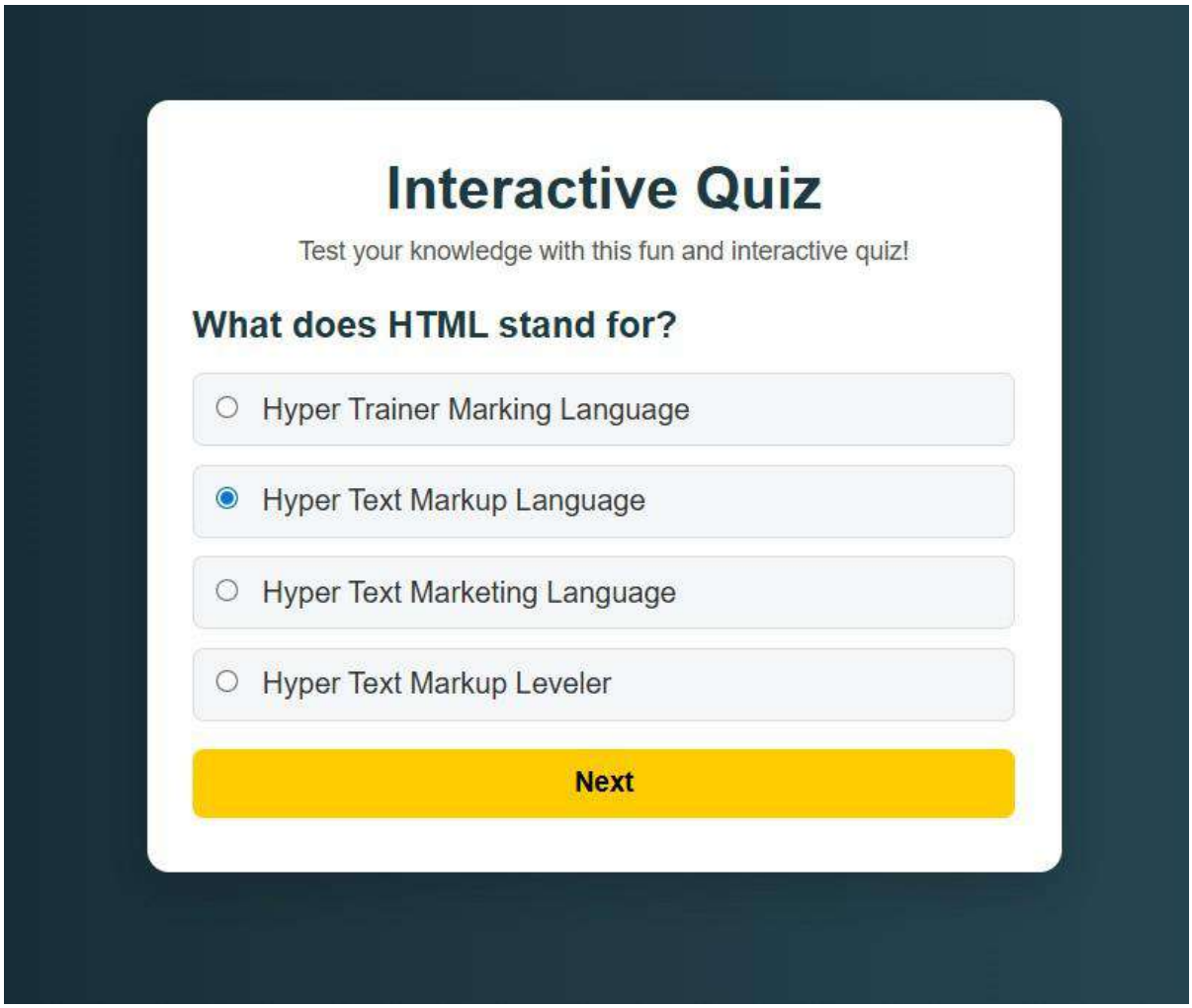
```javascript
const quizData = [
    {
        question: "What does HTML stand for?",
        options: [
            "Hyper Trainer Marking Language",
            "Hyper Text Markup Language",
            "Hyper Text Marketing Language",
            "Hyper Text Markup Leveler"
        ],
        correct: 1
    },
    {
        question: "Which language is used for styling web pages?",
        options: ["HTML", "JQuery", "CSS", "XML"],
        correct: 2
    },
    {
        question: "Which language is used for web interactivity?",
        options: ["Python", "Java", "C++", "JavaScript"],
        correct: 3
    },
    {
        question: "Which tag is used to link CSS to HTML?",
        options: ["<script>", "<style>", "<css>", "<link>"],
        correct: 3
    },
    {
        question: "Which symbol is used for comments in JavaScript?",
        options: ["<!-- -->", "/* */", "//", "#"],
        correct: 2
    }
];
```

```
const questionEl = document.getElementById("question");
const optionEls = document.getElementsByName("option");
const optionTexts = [
    document.getElementById("option0"),
    document.getElementById("option1"),
    document.getElementById("option2"),
    document.getElementById("option3")
];
const nextBtn = document.getElementById("next-btn");
const quizBox = document.getElementById("quiz-box");
const resultBox = document.getElementById("result-box");
const scoreText = document.getElementById("score-text");
const messageText = document.getElementById("message-text");
const restartBtn = document.getElementById("restart-btn");

// Feedback elements
const feedbackText = document.getElementById("feedback-text");
const submitFeedbackBtn = document.getElementById("submit-feedback-btn");
const feedbackMsg = document.getElementById("feedback-msg");

// Quiz state
let currentQuestionIndex = 0;
let score = 0;

// Load question
function loadQuestion() {
    const currentQuestion = quizData[currentQuestionIndex];
    questionEl.textContent = currentQuestion.question;

    currentQuestion.options.forEach((option, index) => {
        optionTexts[index].textContent = option;
        optionEls[index].checked = false;
    });
}
```

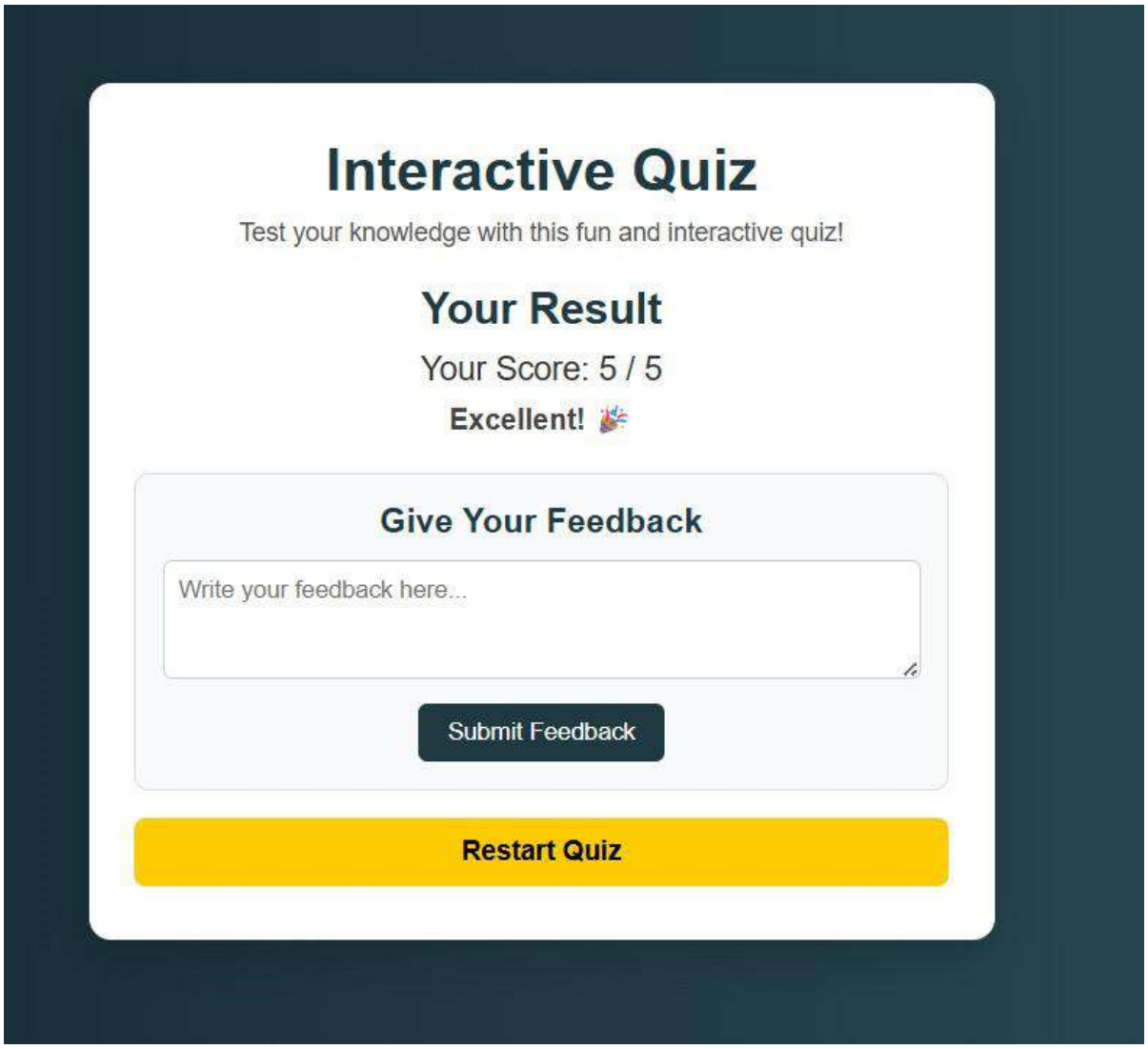
```
function showResult() {
    quizBox.classList.add("hidden");
    resultBox.classList.remove("hidden");

    scoreText.textContent = `Your Score: ${score} / ${quizData.length

    if (score === quizData.length) {
        messageText.textContent = "Excellent! 🎉";
    } else if (score >= quizData.length / 2) {
        messageText.textContent = "Good Job! 👍";
    } else {
        messageText.textContent = "Try Again! 💪";
    }
}

// Next button click
nextBtn.addEventListener("click", () => {
    const selectedAnswer = getSelectedAnswer();

    if (selectedAnswer === null) {
        alert("Please select an option before proceeding!");
        return;
    }

    if (selectedAnswer === quizData[currentQuestionIndex].correct) {
        score++;
    }

    currentQuestionIndex++;

    if (currentQuestionIndex < quizData.length) {
        loadQuestion();
    } else {
        showResult();
    }
});
```

```
// Feedback submit
submitFeedbackBtn.addEventListener("click", () => {
    if (feedbackText.value.trim() === "") {
        alert("Please write some feedback before submitting!");
        return;
    }

    feedbackMsg.textContent = "Thank you for your feedback! 😊";
    feedbackMsg.classList.remove("hidden");
    feedbackText.value = "";
});

// Initial load
loadQuestion();
```

OUTPUT



Interactive Quiz
Test your knowledge with this fun and interactive quiz!
Which tag is used to link CSS to HTML?
<script>
<style>
<css>
<link>
Next



Result:

An interactive quiz application was successfully created using HTML, CSS, and JavaScript. The application is user-friendly, visually appealing, and meets all the experiment requirements including scoring, result display, feedback, and restart functionality.

Experiment No -4

AIM- The aim of this project is to develop a dynamic and interactive To-Do List web application using HTML, CSS, JavaScript, integrated with a Node.js + Express backend and SQLite database. The application allows users to add, edit, delete, mark tasks as completed, filter, search, and sort tasks while ensuring data persistence.

🍊 Project Structure-

```
dynamic_todo_list/
|
|— frontend/
|   |— index.html      # HTML structure for UI
|   |— style.css       # Styling for layout, colors, and responsiveness
|   |— script.js       # JavaScript logic for task management and API calls
|
└— backend/
    |— server.js       # Express server handling API requests
    |— package.json    # Node.js project dependencies
    |— db.sqlite       # SQLite database for task persistence
```

Technologies Used

Technology	Purpose
HTML5	Structure of the web interface
CSS3	Layout, styling, priority indicators, hover effects
JavaScript (ES6)	Task management logic, DOM manipulation, and API calls
Node.js	Backend runtime environment
Express.js	Framework for REST API endpoints
SQLite	Lightweight database for persistent storage
CORS	Middleware to allow frontend-backend communication
Fetch API	Connect frontend with backend APIs

ϟ Features Implemented

Frontend

- Add Tasks: Users can enter a task title and select priority.
- Edit Tasks: Modify task title or priority.
- Delete Tasks: Remove tasks permanently.
- Mark as Completed: Completed tasks are visually differentiated.
- Filter Tasks: View all, active, or completed tasks.
- Search Tasks: Find tasks by keyword (bonus feature).
- Sort Tasks: Sort by priority level (bonus feature).
- Task Counter: Displays total and completed tasks.

Backend

- **Stores all tasks in SQLite database for persistent storage.**
- **Provides RESTful API endpoints for CRUD operations:**
  - **GET /tasks → Fetch all tasks**
  - **POST /tasks → Add new task**
  - **PUT /tasks/:id → Edit task**
  - **PATCH /tasks/:id/status → Toggle task completion**
  - **DELETE /tasks/:id → Delete task**

📖 Implementation Overview

1. **Frontend**
   - **HTML provides the structure, including input fields, buttons, and task list.**
   - **CSS styles the layout, colors, and responsive behavior.**
   - **JavaScript manages tasks dynamically in the browser, interacting with the backend using fetch API.**
2. **Backend**
   - **Node.js + Express handles API requests.**
   - **SQLite database stores tasks permanently.**
   - **Each API endpoint performs specific CRUD actions and sends responses to the frontend.**
3. **Task Lifecycle**
   - **Tasks are created via the Add button → POST request to backend → database stores task → frontend updates list.**
   - **Tasks can be edited, marked completed, deleted, filtered, searched, and sorted, with changes reflected in both UI and database.**

CODE – Frontend

Index.html

```html
<!DOCTYPE html>
<html>
<head>
<title>My To-Do List</title>
<link rel="stylesheet" href="style.css">
</head>

<body>

<h1>My To-Do List</h1>

<div class="input-box">

<input type="text" id="taskTitle" placeholder="Enter task title">

<select id="priority">
<option value="Low">Low</option>
<option value="Medium" selected>Medium</option>
<option value="High">High</option>
</select>

<button onclick="addTask()">Add Task</button>

</div>

<div class="bonus">

<input type="text" id="searchBox" placeholder="Search task..." onkeyup="searchTask()">

<button onclick="sortPriority()">Sort by Priority</button>

</div>

<div class="filters">
```

```html
<button onclick="filterTasks('all')">All</button>
<button onclick="filterTasks('active')">Active</button>
<button onclick="filterTasks('completed')">Completed</button>

</div>

<ul id="taskList"></ul>

<p id="counter"></p>

<script src="script.js"></script>
```

Style.css

```
body {
    background: linear-gradient(135deg, #ffe6e6, #fff0f5);
    text-align: center;
    margin: 0;
    padding: 0;
}

/* Heading styling */
h1 {
    margin-top: 30px;
    font-size: 2em;
    color: #333;
}

/* Input and select styling */
input, select {
    padding: 10px 12px;
    margin: 8px 5px;
    border: 1px solid #ccc;
    border-radius: 5px;
    font-size: 1em;
    outline: none;
    transition: 0.3s;
}

input:focus, select:focus {
    border-color: #007bff;
    box-shadow: 0 0 5px rgba(0,123,255,0.3);
}

/* Button styling */
button {
    padding: 10px 16px;
    margin: 8px 5px;
    cursor: pointer;
    border: none;
    border-radius: 5px;
```

```
/* List styling */
ul {
    padding: 0;
    margin: 20px auto;
    max-width: 450px;
}

li {
    list-style: none;
    background: white;
    margin: 10px 0;
    padding: 12px 15px;
    border-radius: 8px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    box-shadow: 0 2px 6px rgba(0,0,0,0.1);
    transition: 0.3s;
}

li:hover {
    transform: translateY(-2px);
    box-shadow: 0 4px 10px rgba(0,0,0,0.15);
}

/* Completed task styling */
.completed {
    text-decoration: line-through;
    color: gray;
    opacity: 0.7;
}

/* Priority colors */
.high {
    color: #e74c3c; /* bright red */
    font-weight: bold;
}
```

```
.medium {
    color: #f39c12; /* orange */
    font-weight: bold;
}

.low {
    color: #2ecc71; /* green */
    font-weight: bold;
}
```

Script.js

```
let tasks=[]

function loadTasks(){

fetch("http://localhost:3000/tasks")
.then(res=>res.json())
.then(data=>{
tasks=data
displayTasks(tasks)
})

}

function addTask(){

let title=document.getElementById("taskTitle").value
let priority=document.getElementById("priority").value

if(title=="" || title.length>50){
alert("Task title invalid")
return
}
```

```
function displayTasks(list){
    let ul=document.getElementById("taskList")
    ul.innerHTML=""

    list.forEach(task=>{
        let li=document.createElement("li")

        if(task.isDone){
            li.classList.add("completed")
        }

        li.innerHTML=`
        <span class="${task.priority.toLowerCase()} task-text">
            ${task.title} (${task.priority})
        </span>

        <div>
            <button onclick="toggleTask(${task.id})">✓</button>
            <button onclick="editTask(${task.id}, '${task.title}', '${task.priority}')">✏️</button>
            <button onclick="deleteTask(${task.id})">❌</button>
        </div>
        `


        ul.appendChild(li)
    })

    updateCounter()
}
function editTask(id, currentTitle, currentPriority){

    let newTitle = prompt("Edit Task Title:", currentTitle)
    if(!newTitle || newTitle.trim()=="" || newTitle.length>50) return

    let newPriority = prompt("Edit Priority (Low / Medium / High):", currentPriority)
    if(!["Low","Medium","High"].includes(newPriority)) return
```

```
function toggleTask(id){

fetch(`http://localhost:3000/tasks/${id}/status`,{
method:"PATCH"
})
.then(()=>loadTasks())

}

function deleteTask(id){
```

```
function deleteTask(id){

fetch(`http://localhost:3000/tasks/${id}`,{
method:"DELETE"
})
.then(()=>loadTasks())

}

function filterTasks(type){

if(type=="all")
displayTasks(tasks)

if(type=="active")
displayTasks(tasks.filter(t=>t.isDone==0))

if(type=="completed")
displayTasks(tasks.filter(t=>t.isDone==1))

}

function updateCounter(){

let total=tasks.length
let completed=tasks.filter(t=>t.isDone==1).length

document.getElementById("counter").innerText=
`Completed ${completed} / ${total}`

}

/* BONUS FEATURE 1 */
function searchTask(){

let keyword=document.getElementById("searchBox").value.toLowerCase()
```

```
let filtered=tasks.filter(task =>
task.title.toLowerCase().includes(keyword)
)

displayTasks(filtered)

}

/* BONUS FEATURE 2 */
function sortPriority(){

let order={
High:1,
Medium:2,
Low:3
}

let sorted=[...tasks].sort((a,b)=>{
return order[a.priority]-order[b.priority]
})

displayTasks(sorted)

}

loadTasks()
```

Backend – server.js

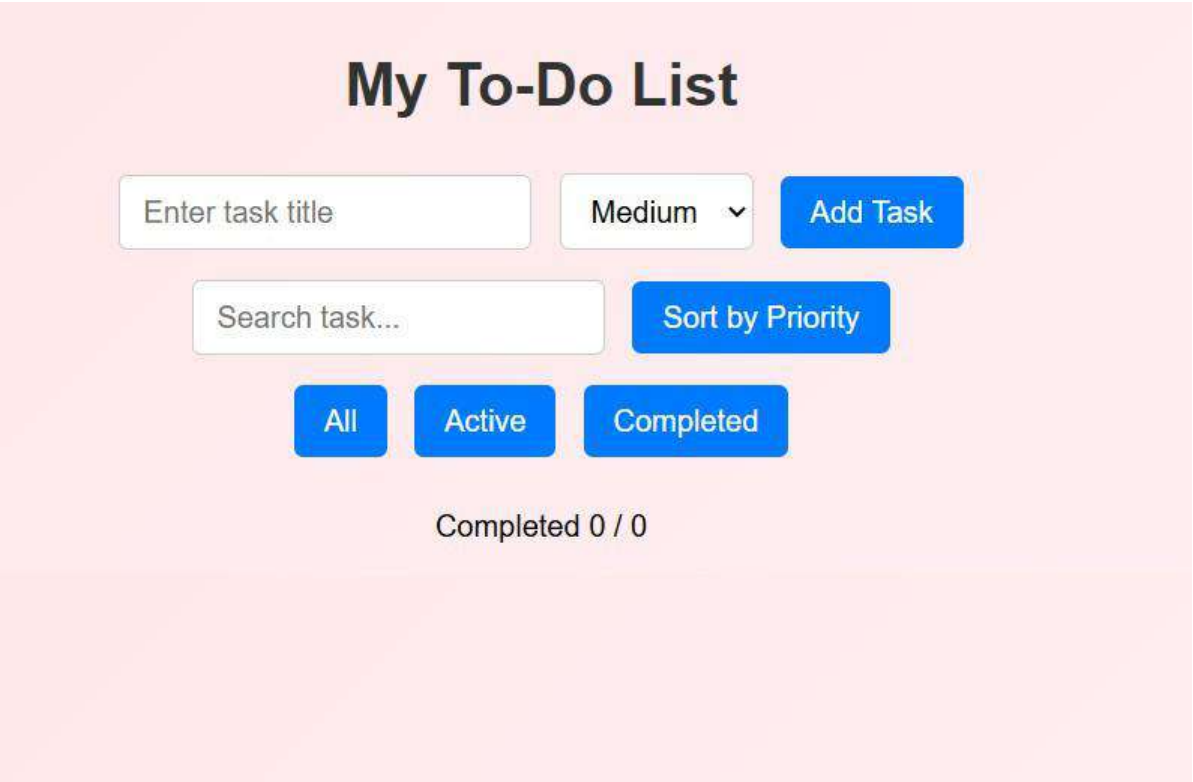
```
const express=require("express")
const sqlite3=require("sqlite3").verbose()
const cors=require("cors")

const app=express()

app.use(cors())
app.use(express.json())

const db=new sqlite3.Database("db.sqlite")

db.run(`
CREATE TABLE IF NOT EXISTS tasks(
id INTEGER PRIMARY KEY AUTOINCREMENT,
title TEXT NOT NULL,
priority TEXT DEFAULT 'Medium',
isDone INTEGER DEFAULT 0,
createdAt DATETIME DEFAULT CURRENT_TIMESTAMP
)
`)

app.get("/tasks",(req,res)=>{
db.all("SELECT * FROM tasks",(err,rows)=>{
res.json(rows)
})
})

app.post("/tasks",(req,res)=>{
const {title,priority}=req.body

db.run(
"INSERT INTO tasks(title,priority) VALUES (?,?)",
[title,priority],
function(){
res.json({id:this.lastID})
})
})
```

```
app.put("/tasks/:id",(req,res)=>{
const {title,priority}=req.body

db.run(
"UPDATE tasks SET title=?,priority=? WHERE id=?",
[title,priority,req.params.id],
()=>res.json({message:"updated"})
)
})

app.patch("/tasks/:id/status",(req,res)=>{
db.run(
"UPDATE tasks SET isDone = NOT isDone WHERE id=?",
[req.params.id],
()=>res.json({message:"status changed"})
)
})

app.delete("/tasks/:id",(req,res)=>{
db.run(
"DELETE FROM tasks WHERE id=?",
[req.params.id],
()=>res.json({message:"deleted"})
)
})

app.listen(3000,()=>{
console.log("Server running on port 3000")
})
```

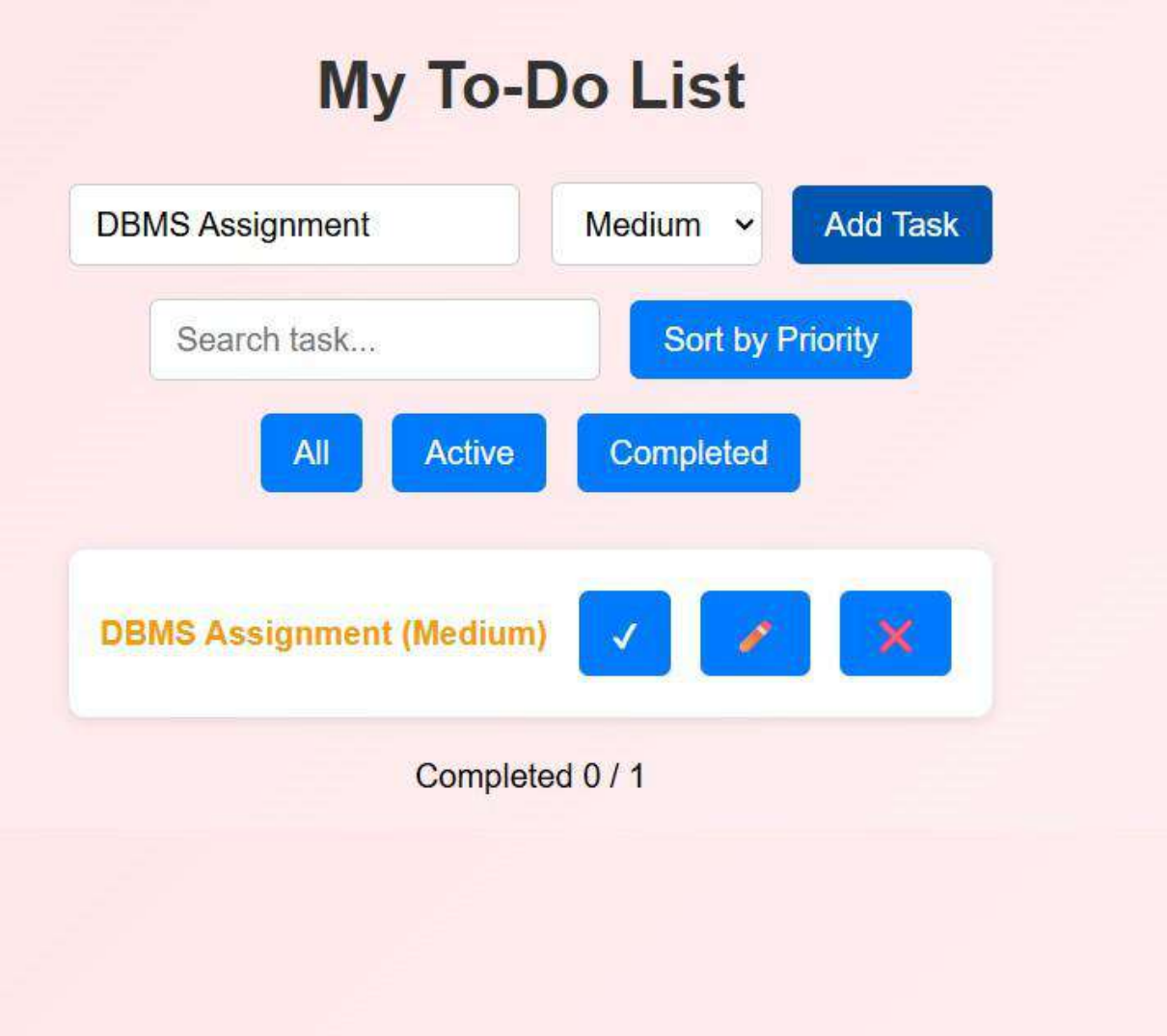
Package.json

```
{
"name":"todo-backend",
"version":"1.0.0",
"main":"server.js",
▷Debug
"scripts":{
"start":"node server.js"
},
"dependencies":{
"express":"^4.18.2",
"sqlite3":"^5.1.6",
"cors":"^2.8.5"
}
}
```

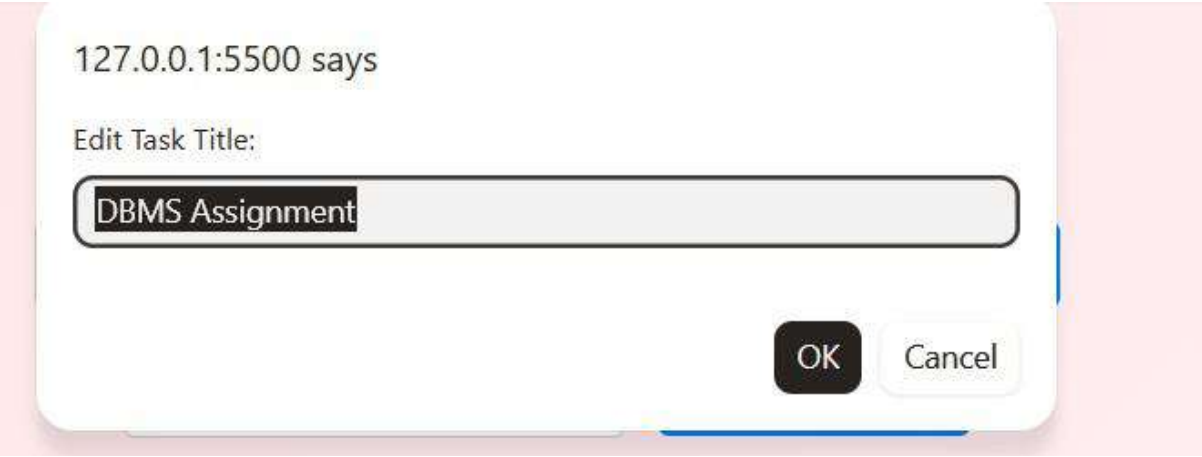
OUTPUT

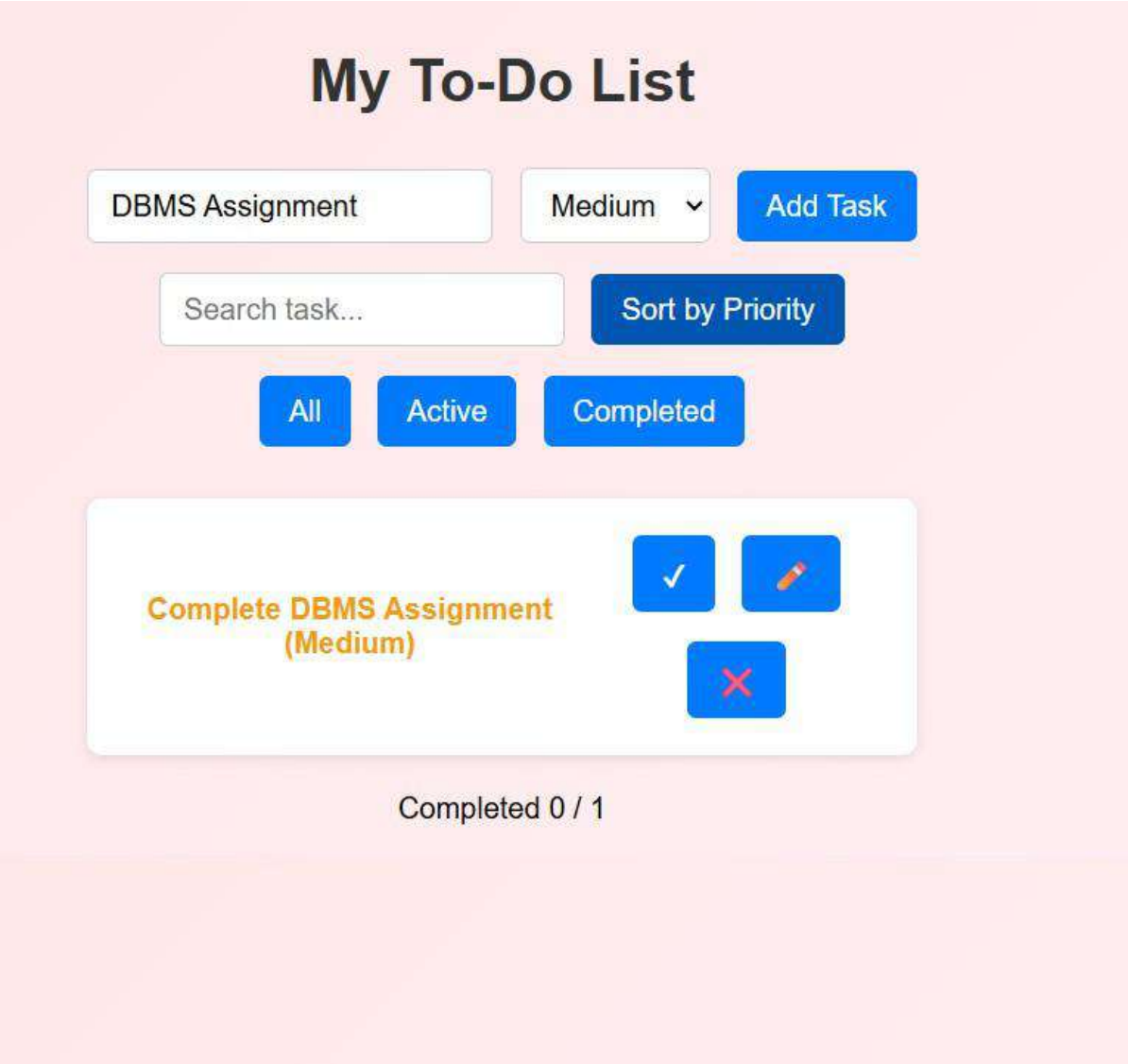


+ **Add Task**

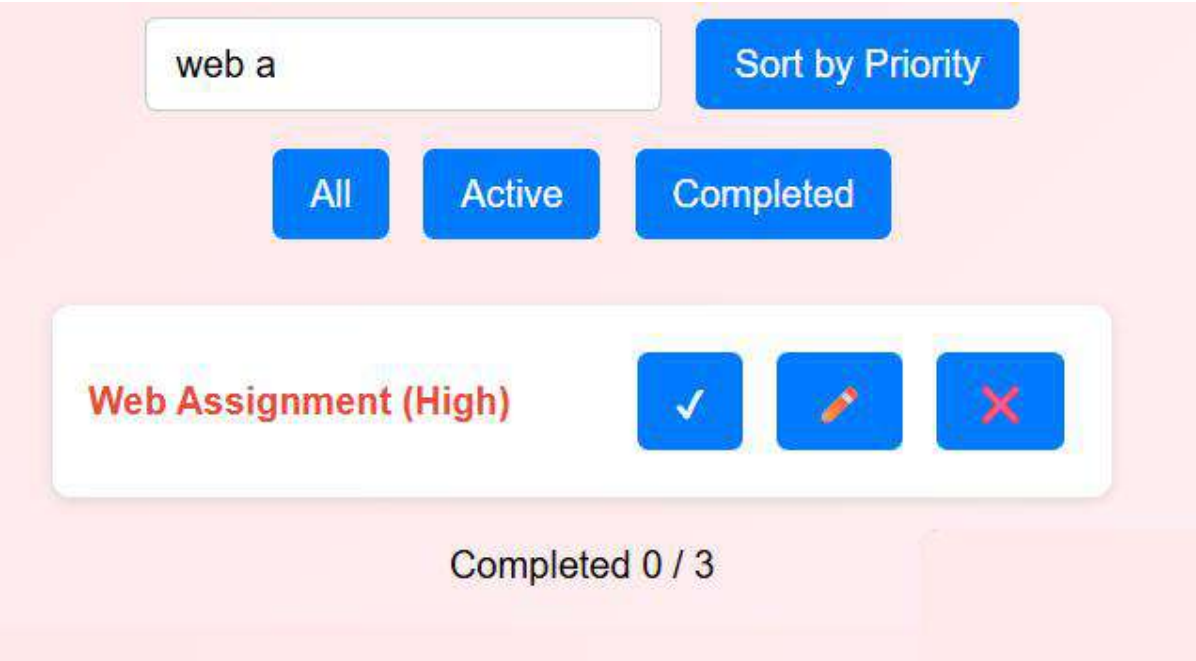


- **Edit Task**

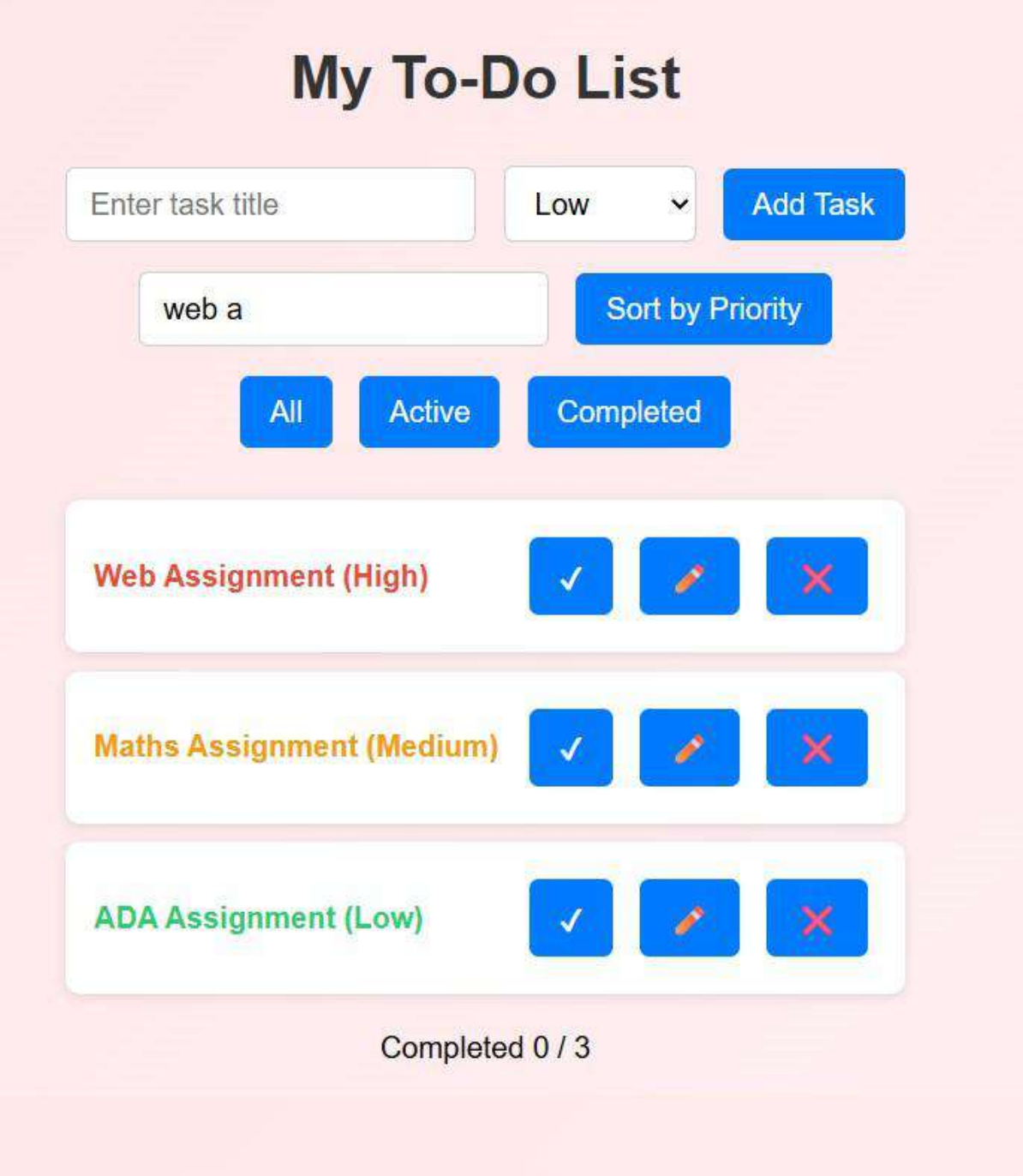



My To-Do List
DBMS Assignment
Medium
Add Task
Search task...
Sort by Priority
All
Active
Completed
Complete DBMS Assignment (Medium)
Completed 0 / 1

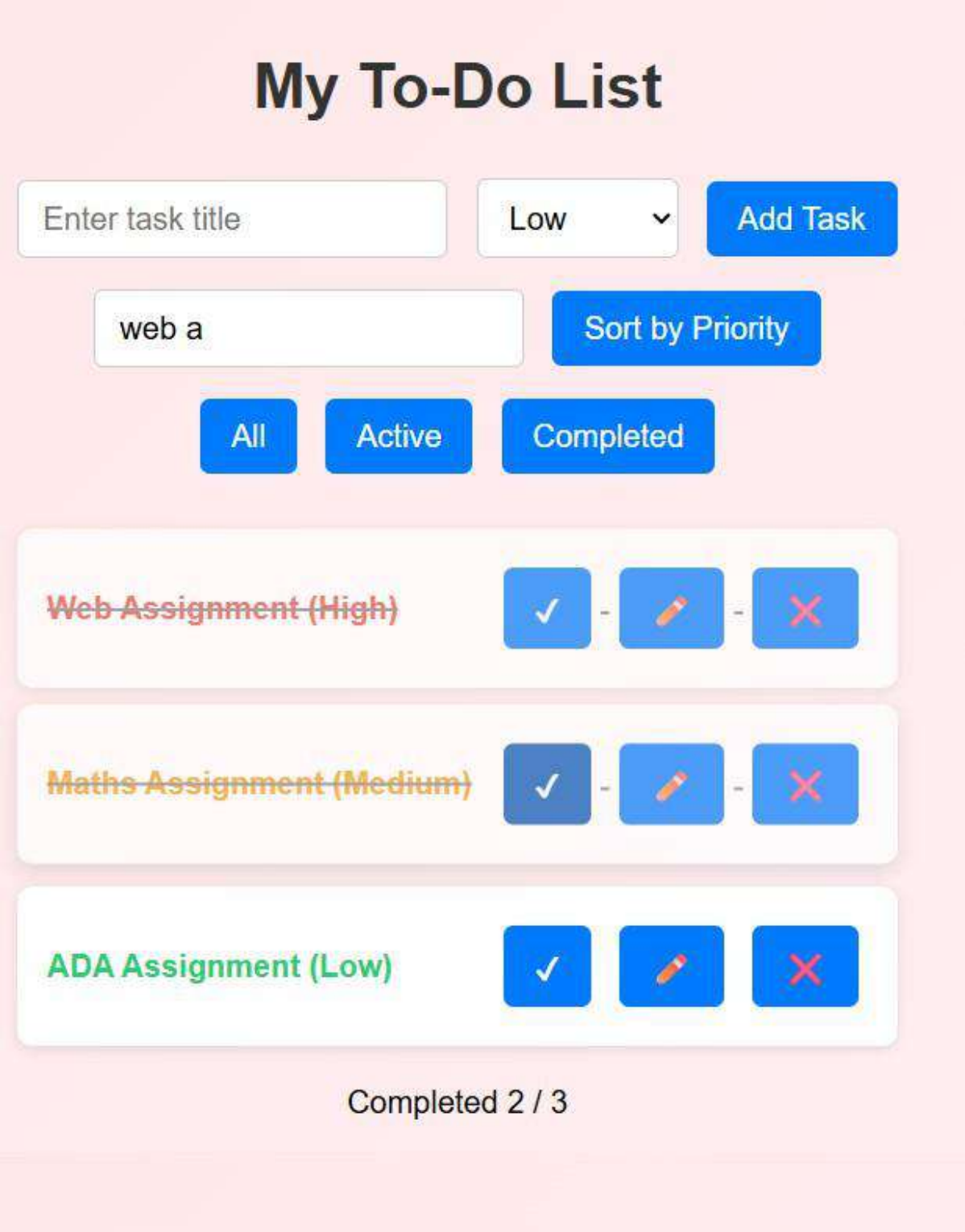
Search by Keyword



Sort by Priority



Complete Task



Conclusion

The Dynamic To-Do List application demonstrates full-stack development by integrating frontend interactivity with backend persistence.
Students learn practical skills in HTML, CSS, JavaScript, Node.js, Express, and SQLite while implementing a real-world web application with CRUD functionality.

🎯 Learning Outcomes

By completing this project, students will be able to:

- **Design a responsive UI using HTML and CSS.**
- **Implement DOM-based CRUD operations using JavaScript.**
- **Use REST API endpoints to perform CRUD operations with a database.**
- **Validate user input and handle edge cases (e.g., empty or too long task titles).**
- **Understand the difference between in-memory storage and persistent storage.**
- **Implement bonus features like search and sort.**

EXPERIMENT-5

AIM - To develop a simple blog using Flask framework that performs CRUD operations (Create, Read, Update, Delete) and displays blog posts using HTML templates with basic styling.

Objective

- **To understand the basics of Flask web framework**
- **To implement CRUD operations without using a database**
- **To learn template rendering using Jinja2**
- **To design a simple and user-friendly interface using CSS**

Requirements

- **Python**
- **Flask**
- **VS Code / Any IDE**
- **Web Browser**

Theory

Flask is a lightweight Python web framework used for building web applications. It allows developers to create web apps quickly with minimal code.

In this experiment, a simple blog system is created where:

- **Users can create blog posts**
- **View all posts**
- **Edit existing posts**
- **Delete posts**

Procedure

1. **Create a project folder named simple_blog**
2. **Install Flask using pip**
3. **Create required files and folders**
4. **Initialize Flask application in app.py**
5. **Create routes for:**
    - **Home page (Read)**
    - **Create post**
    - **Edit post**
    - **Delete post**

6. **Design HTML templates using Jinja2**
7. **Add CSS for styling**
8. **Run the Flask application**
9. **Test all functionalities**

Implementation

1. Create Operation

Users can create a new blog post by entering title and content using a form.

2. Read Operation

All blog posts are displayed on the home page.

3. Update Operation

Users can edit existing posts with pre-filled data.

4. Delete Operation

Users can delete any blog post.

Features

- **Simple and clean UI**
- **CRUD operations implemented**
- **Navigation between pages**
- **Responsive and centered form**
- **Styled using CSS**

CODE

App.py

app.py > ...

```python
# ==============================
# Project: Simple Blog (Experiment-5)
# Name: Ananya Sharma
# Date: 2026-04-05
# ==============================

from flask import Flask, render_template, request, redirect, url_for

app = Flask(__name__)

# In-memory storage
posts = []

# ------------------------------
# Home (Read)
# ------------------------------
@app.route('/')
def index():
    return render_template('index.html', posts=posts)

# ------------------------------
# Create
# ------------------------------
@app.route('/create', methods=['GET', 'POST'])
def create():
    if request.method == 'POST':
        title = request.form['title']
        content = request.form['content']

        posts.append({
            'title': title,
            'content': content
        })

        return redirect(url_for('index'))

    return render_template('create.html')
```

```python
    return render_template('create.html')

# ------------------------------
# Update
# ------------------------------
@app.route('/edit/<int:index>', methods=['GET', 'POST'])
def edit(index):
    post = posts[index]

    if request.method == 'POST':
        post['title'] = request.form['title']
        post['content'] = request.form['content']

        return redirect(url_for('index'))

    return render_template('edit.html', post=post, index=index)

# ------------------------------
# Delete
# ------------------------------
@app.route('/delete/<int:index>')
def delete(index):
    posts.pop(index)
    return redirect(url_for('index'))

# ------------------------------
# Run App
# ------------------------------
if __name__ == '__main__':
    app.run(debug=True)
```

HTML CODE

Base.html

```html
<!DOCTYPE html>
<html>
<head>
    <title>Simple Blog</title>

    <!-- CSS -->
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>

<nav>
    <h2>Simple Blog</h2>
    <div>
        <a href="/">Home</a>
        <a href="/create">Create Post</a>
    </div>
</nav>

<div class="container">
    {% block content %}{% endblock %}
</div>

</body>
</html>
```

Index.html

```html
{% extends 'base.html' %}

{% block content %}

<h2>All Blog Posts</h2>

{% for post in posts %}
<div class="post">
    <h3>{{ post.title }}</h3>
    <p>{{ post.content }}</p>

    <div class="actions">
        <a class="edit" href="/edit/{{ loop.index0 }}">Edit</a>
        <a class="delete" href="/delete/{{ loop.index0 }}">Delete</a>
    </div>
</div>

{% else %}
<p class="empty">No posts available. Create one!</p>
{% endfor %}

{% endblock %}
```

Create.html

```html
{% extends 'base.html' %}

{% block content %}

<h2>Create New Post</h2>

<form method="POST" class="form">
    <label>Title</label>
    <input type="text" name="title" placeholder="Enter title" required>

    <label>Content</label>
    <textarea name="content" placeholder="Enter content" required></textarea>

    <button type="submit">Add Post</button>
</form>

{% endblock %}
```

Edit.html

```html
{% extends 'base.html' %}

{% block content %}

<h2>Edit Post</h2>

<form method="POST" class="form">
    <label>Title</label>
    <input type="text" name="title" value="{{ post.title }}" required>

    <label>Content</label>
    <textarea name="content" required>{{ post.content }}</textarea>

    <button type="submit">Update Post</button>
</form>

{% endblock %}
```

CSS – style.css

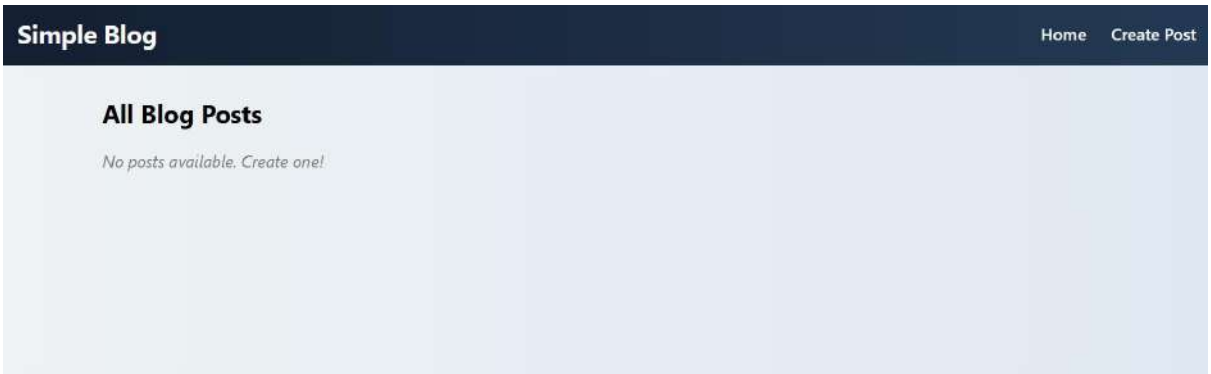
```css
/* Background */
body {
    font-family: 'Segoe UI', sans-serif;
    background: linear-gradient(to right, #eef2f3, #dfe9f3);
    margin: 0;
}

/* Navbar */
nav {
    background: linear-gradient(90deg, #141e30, #243b55);
    color: white;
    padding: 15px 40px;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

nav h2 {
    margin: 0;
}

nav a {
    color: white;
    margin-left: 20px;
    text-decoration: none;
    font-weight: 500;
}

nav a:hover {
    color: #00e6e6;
}

/* Container */
.container {
    width: 80%;
    margin: 30px auto;
}
```

```
}

.post h3 {
    margin-top: 0;
}

/* Buttons */
.actions {
    margin-top: 10px;
}

.actions a {
    padding: 6px 12px;
    border-radius: 6px;
    text-decoration: none;
    margin-right: 10px;
    font-size: 14px;
}

.edit {
    background-color: #28a745;
    color: white;
}

.delete {
    background-color: #e74c3c;
    color: white;
}

/* CENTER FORM ✨ */
.form {
    width: 50%;
    margin: 40px auto;   /* Center horizontally */
    background: white;
    padding: 25px;
    border-radius: 12px;
    box-shadow: 0 4px 15px rgba(0,0,0,0.1);
```

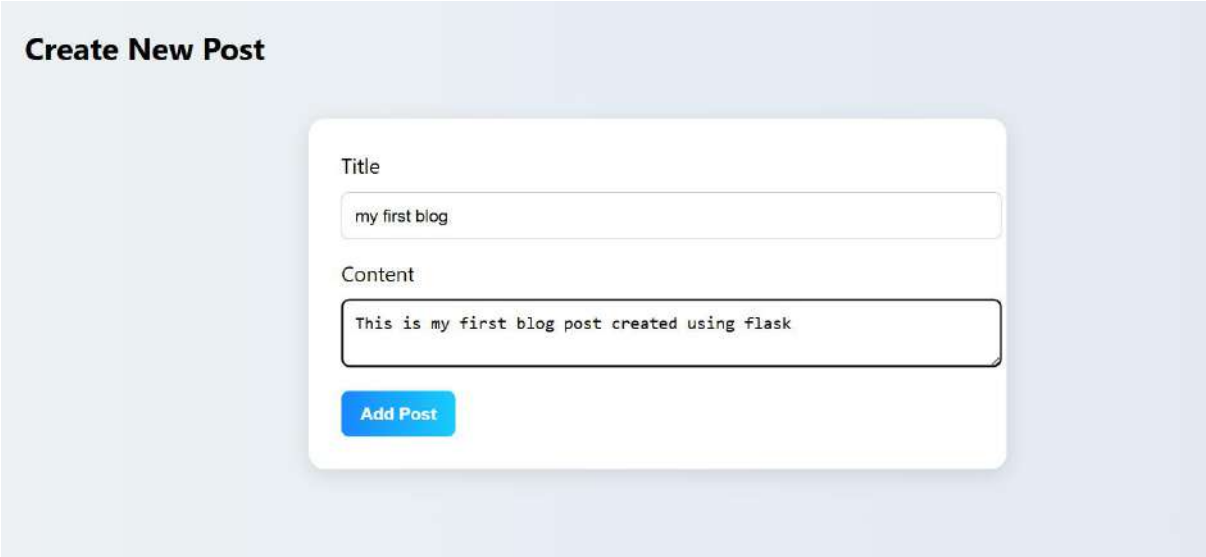
```css
/* Inputs */
input, textarea {
    width: 100%;
    padding: 10px;
    margin: 10px 0 15px 0;
    border-radius: 6px;
    border: 1px solid #ccc;
}

/* Button */
button {
    background: linear-gradient(90deg, #007bff, #00c6ff);
    color: white;
    border: none;
    padding: 10px 15px;
    border-radius: 6px;
    cursor: pointer;
    font-weight: bold;
}

button:hover {
    opacity: 0.9;
}

/* Empty text */
.empty {
    color: #777;
    font-style: italic;
}
```

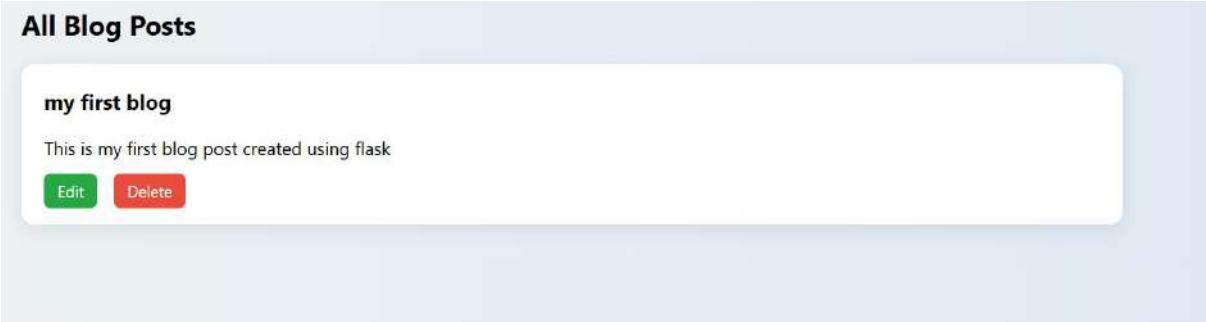
Outputs

Home Page

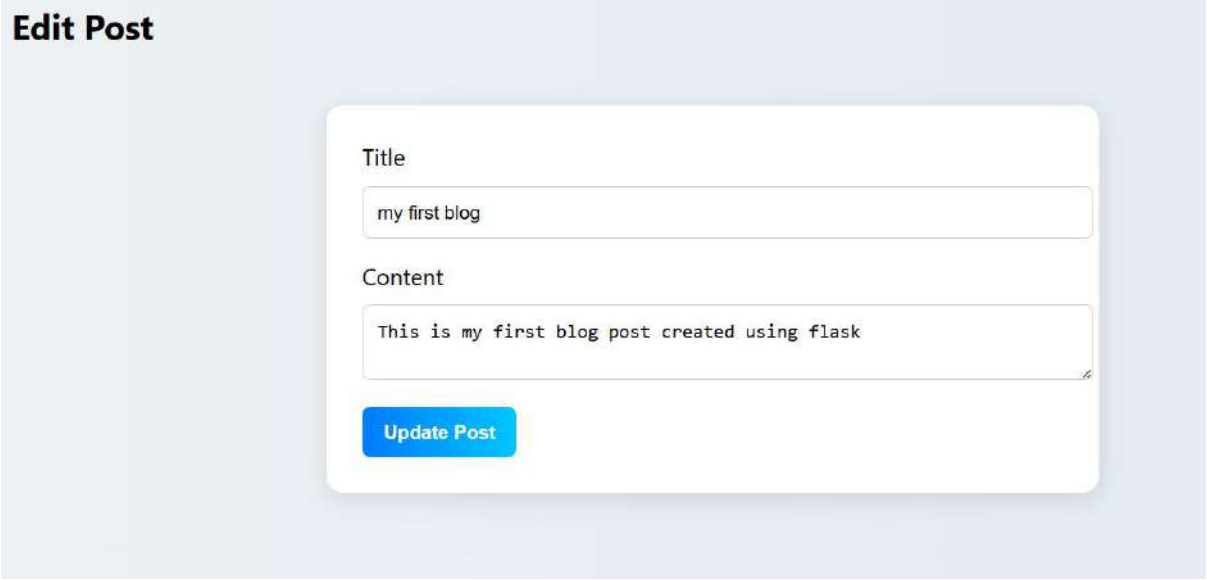


Create post form

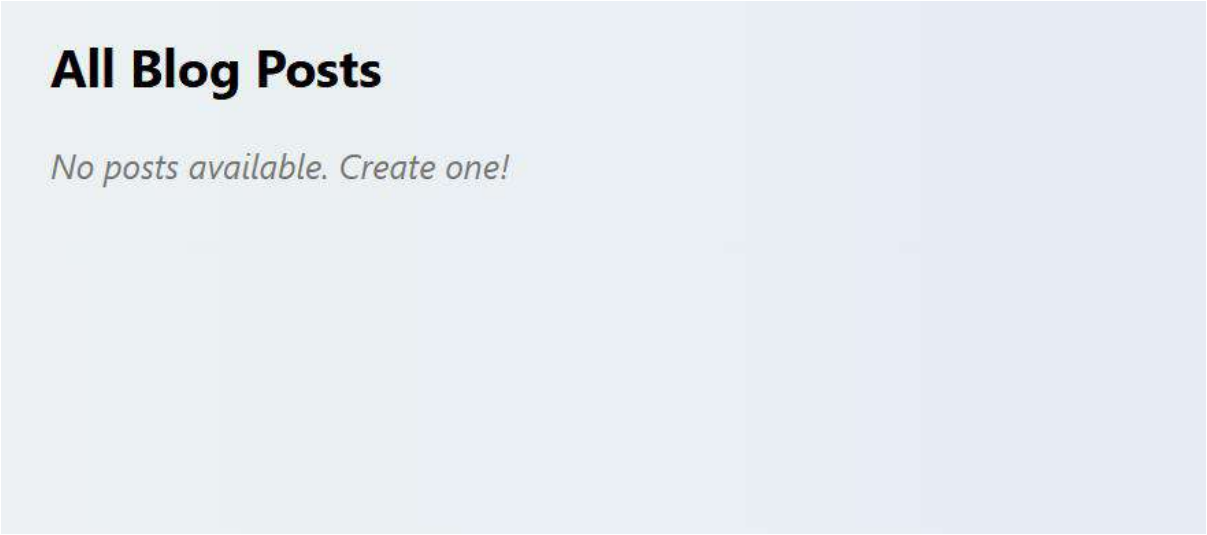




Edit Post Functionality



After Using Delete Option



☑ Result

The simple blog application was successfully created using Flask with full CRUD functionality and a user-friendly interface.

EXPERIMENT– 6

AIM -To develop a web-based Contact Management System using Flask that performs CRUD (Create, Read, Update, Delete) operations.

Objective

- **To understand the basics of Flask framework**
- **To implement CRUD operations without using a database**
- **To design a simple and user-friendly interface using HTML and CSS**

Technology Used

- **Python**
- **Flask**
- **HTML**
- **CSS**

📚 Theory

Flask is a lightweight web framework in Python used to build web applications. It allows developers to create dynamic web pages using templates.

CRUD operations are basic functions used in applications:

- **Create → Add new data**
- **Read → Display data**
- **Update → Modify existing data**
- **Delete → Remove data**
- **In this project, contacts are stored in a list (temporary storage) and managed using Flask routes.**

❁ Procedure

1. Create a project folder named contact_management_system
2. Install Flask using pip
3. Create required files:
   - app.py
   - templates (HTML files)
   - static (CSS file)
4. Initialize Flask application
5. Create routes for:
   - Home page (display contacts)
   - Add contact
   - Edit contact
   - Delete contact
6. Design HTML templates for UI
7. Apply CSS for styling
8. Run the application and test all features

💻 Code Description

- app.py handles all backend logic and routes
- Contacts are stored in a list
- HTML templates display data dynamically
- CSS improves the appearance of the application

Features

- Add new contact
- View all contacts
- Update contact details
- Delete contact
- Search contact
- User-friendly interface with styling

Explanation

◇ Add Contact (Create)

The user enters name, phone, and email through a form.
This data is stored in the contacts list.

◇ View Contacts (Read)

All contacts are displayed in a table format.

The user can easily view all stored details.

- **Edit Contact (Update)**

Existing contact details are pre-filled in the form.
The user can update and save the changes.

- **Delete Contact (Delete)**

The user can delete any contact from the list.
The selected contact is permanently removed.

- **Search Contact**

The user can search contacts by name or phone number.
Only matching results are displayed.

- **Frontend (HTML & CSS)**

HTML templates are used to design the interface.
CSS is applied to improve the appearance.

- **Backend (Flask)**

Flask routes handle different operations.
The app.py file controls the application logic.

CODE

App.py

```
from flask import Flask, render_template, request, redirect, url_for

app = Flask(__name__)

contacts = []

@app.route('/')
def index():
    query = request.args.get('search')
    if query:
        filtered = [c for c in contacts if query.lower() in c['name'].lower() or query in c['phone']]
    else:
        filtered = contacts
    return render_template('index.html', contacts=filtered, query=query)

@app.route('/add', methods=['GET', 'POST'])
def add_contact():
    if request.method == 'POST':
        name = request.form['name']
        phone = request.form['phone']
        email = request.form['email']

        if not name or not phone or not email:
            return "All fields are required!"
```

```python
        contacts.append({'name': name, 'phone': phone, 'email': email})
        return redirect(url_for('index'))
    return render_template('add_contact.html')

@app.route('/edit/<int:id>', methods=['GET', 'POST'])
def edit_contact(id):
    if request.method == 'POST':
        contacts[id]['name'] = request.form['name']
        contacts[id]['phone'] = request.form['phone']
        contacts[id]['email'] = request.form['email']
        return redirect(url_for('index'))
    return render_template('edit_contact.html', contact=contacts[id], id=id)

@app.route('/delete/<int:id>')
def delete_contact(id):
    contacts.pop(id)
    return redirect(url_for('index'))

if __name__ == '__main__':
    app.run(debug=True)
```

HTML CODE

Base.html

```html
<!DOCTYPE html>
<html>
<head>
    <title>Contact Manager</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>

<div class="navbar">
    <h2>Contact Manager</h2>
    <div>
        <a href="/" class="btn">Home</a>
        <a href="/add" class="btn">Add Contact</a>
    </div>
</div>

<div class="container">
    {% block content %}{% endblock %}
</div>

</body>
</html>
```

Index.html

```html
<h3 class="center">Your Contacts</h3>

<form method="GET">
    <div class="search-box">
        <input type="text" name="search" placeholder="Search by name or phone" value="{{ query if query else '' }}">
        <button class="btn" type="submit">Search</button>
    </div>
</form>

<br>

<table>
<tr>
    <th>Name</th>
    <th>Phone</th>
    <th>Email</th>
    <th>Actions</th>
</tr>

{% for contact in contacts %}
<tr>
    <td>{{ contact.name }}</td>
    <td>{{ contact.phone }}</td>
    <td>{{ contact.email }}</td>
    <td>
        <a href="/edit/{{ loop.index0 }}" class="btn">Edit</a>
        <a href="/delete/{{ loop.index0 }}" class="btn">Delete</a>
    </td>
</tr>
{% endfor %}
</table>

{% if contacts|length == 0 %}
    <p class="center">No contacts found 🤔</p>
{% endif %}
```

Add_contacts.html

```html
{% extends 'base.html' %}
{% block content %}

<h3 class="center">Add Contact</h3>

<form method="POST">
    <input type="text" name="name" placeholder="Enter Name">
    <input type="text" name="phone" placeholder="Enter Phone">
    <input type="email" name="email" placeholder="Enter Email">
    <button class="btn" type="submit">Add Contact</button>
</form>

{% endblock %}
```

Edit_contacts.html

```html
{% extends 'base.html' %}
{% block content %}

<h3 class="center">Edit Contact</h3>

<form method="POST">
    <input type="text" name="name" value="{{ contact.name }}">
    <input type="text" name="phone" value="{{ contact.phone }}">
    <input type="email" name="email" value="{{ contact.email }}">
    <button class="btn" type="submit">Update Contact</button>
</form>

{% endblock %}
```

Style.css

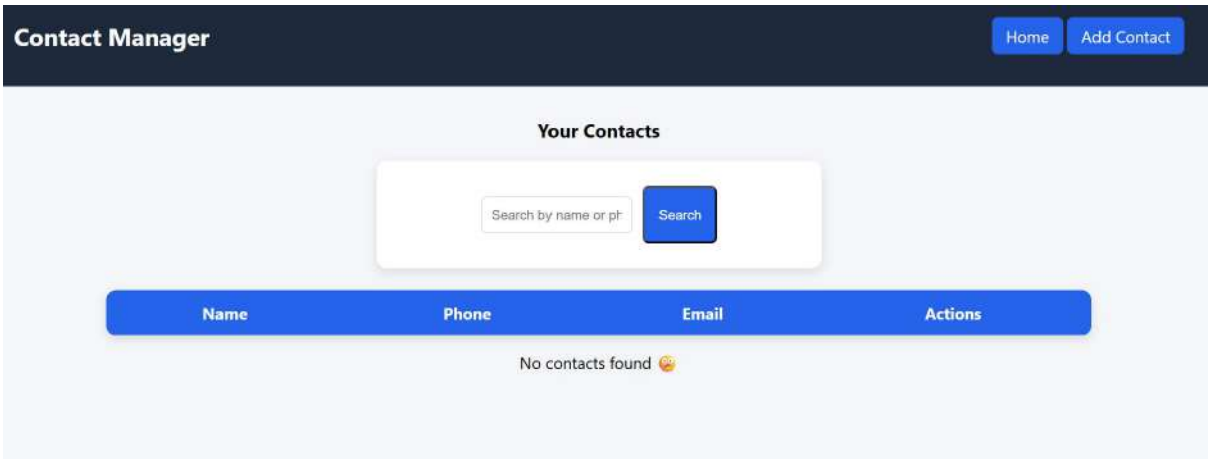
```css
body {
    font-family: 'Segoe UI', sans-serif;
    margin: 0;
    background: #f4f6f9;
}

/* Navbar */
.navbar {
    background: #1e293b;
    color: white;
    padding: 15px 30px;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

/* Container */
.container {
    width: 80%;
    margin: 30px auto;
}
```

```css
/* Buttons */
.btn {
    padding: 8px 14px;
    background: #2563eb;
    color: white;
    border-radius: 6px;
    text-decoration: none;
    transition: 0.3s;
}

.btn:hover {
    background: #1e40af;
    transform: scale(1.05);
}

/* Table */
table {
    width: 100%;
    border-collapse: collapse;
    background: white;
    border-radius: 10px;
    overflow: hidden;
    box-shadow: 0 4px 10px rgba(0,0,0,0.1);
}

th, td {
    padding: 12px;
    border-bottom: 1px solid #ddd;
    text-align: center;
}

th {
    background: #2563eb;
    color: white;
}
```

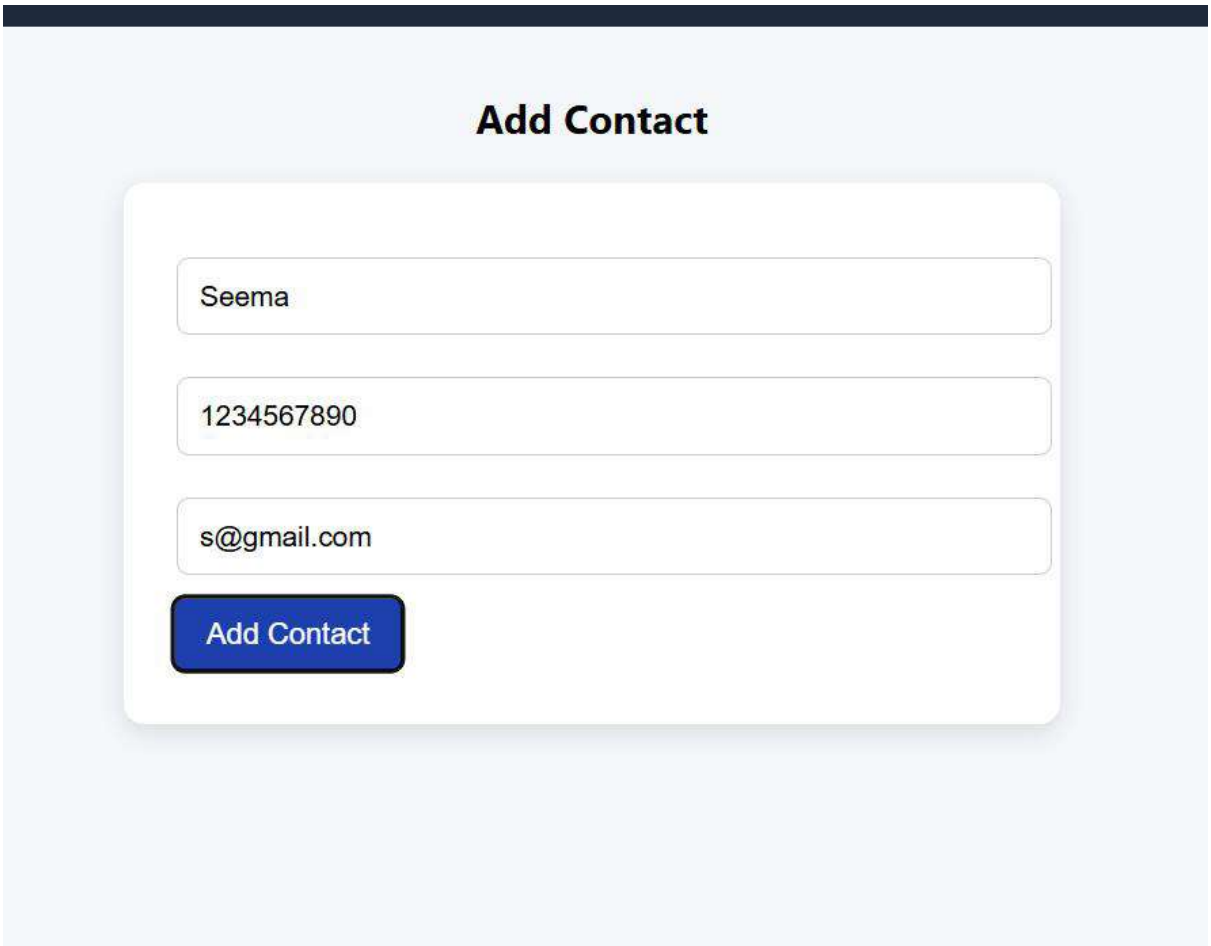
```css
form {
    background: white;
    padding: 25px;
    border-radius: 10px;
    width: 40%;
    margin: auto;
    box-shadow: 0 4px 10px rgba(0,0,0,0.1);
}

input {
    width: 100%;
    padding: 10px;
    margin: 10px 0;
    border-radius: 6px;
    border: 1px solid #ccc;
}

input:focus {
    border-color: #2563eb;
    outline: none;
}

/* Search */
.search-box {
    width: 60%;
    margin: auto;
    display: flex;
    gap: 10px;
}

.search-box input {
    flex: 1;
}

/* Center */
.center {
```

OUTPUTS

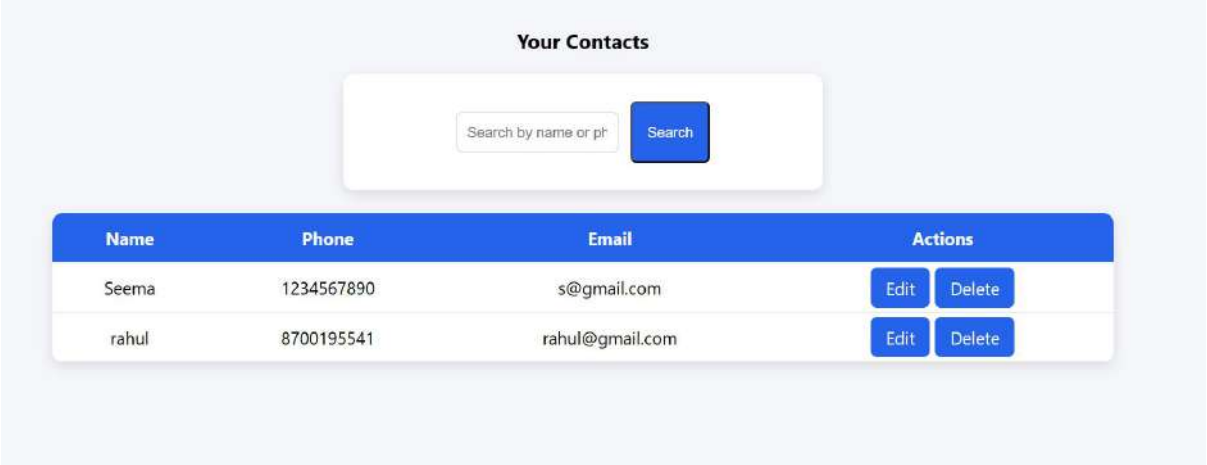
Home page



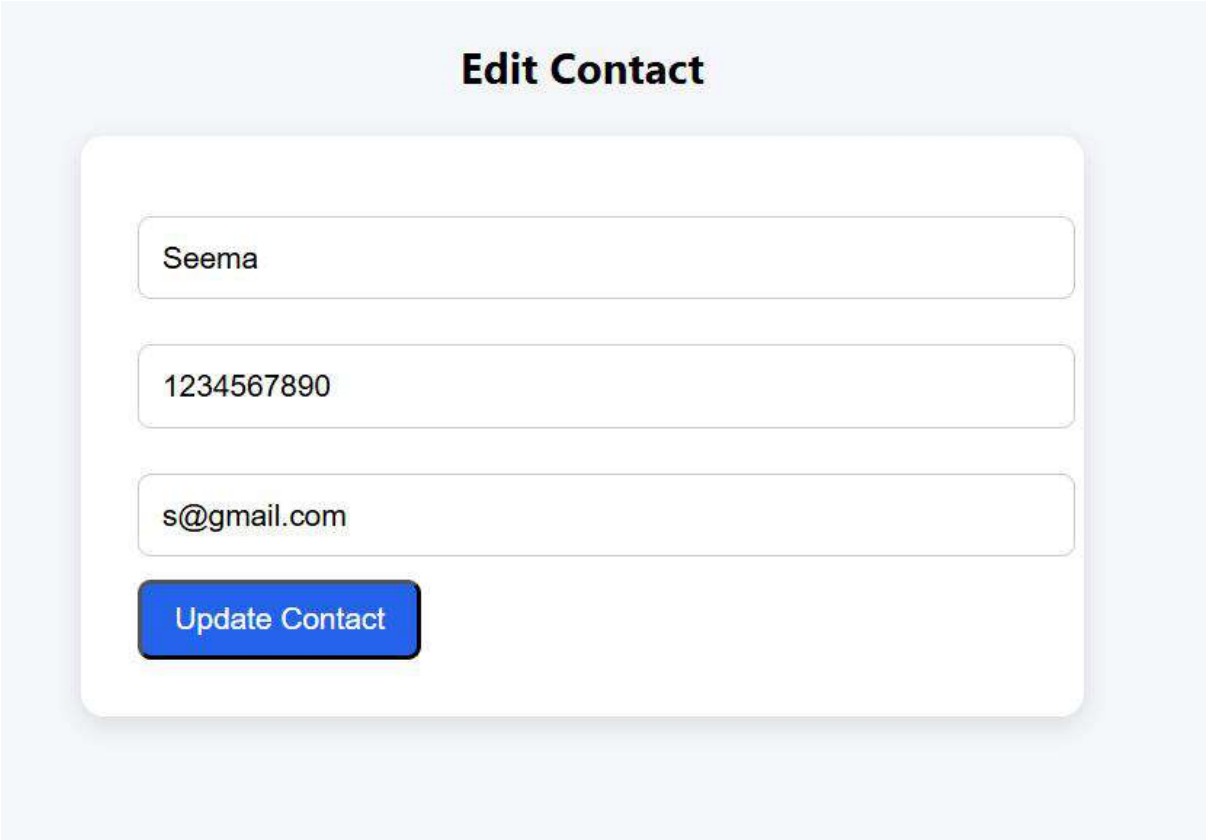
Add contact form



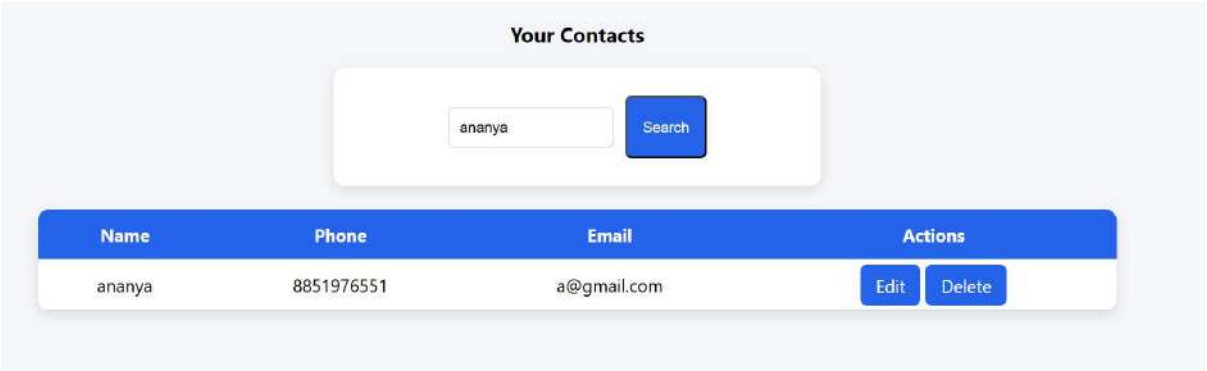
View contacts



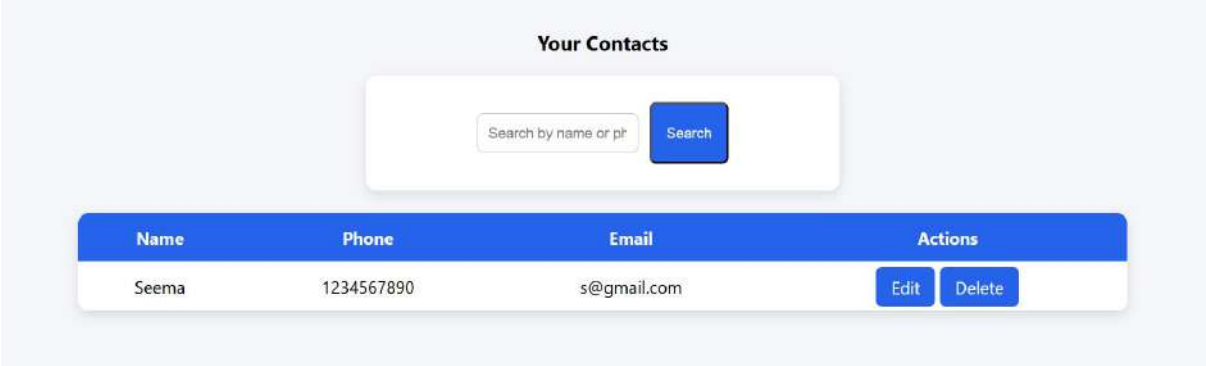
Edit contact form



Search contacts



Delete contacts



☑ Result

The Contact Management System was successfully developed using Flask. All CRUD operations are working properly, and the interface is simple and user-friendly.

EXPERIMENT – 7

AIM-To design and develop a simple e-commerce website using HTML, CSS, and JavaScript with product listing, shopping cart functionality, and checkout simulation.

Objective

- **To understand frontend web development**
- **To implement dynamic cart functionality using JavaScript**
- **To design a user-friendly interface using CSS**

🛠 Tools & Technologies Used:

- **HTML**
- **CSS**
- **JavaScript**
- **Web Browser**

📚 Theory

An e-commerce website allows users to browse products, add them to a cart, and proceed to checkout.

- **HTML is used to structure the webpage**
- **CSS is used for styling and layout**
- **JavaScript is used to handle dynamic operations like adding items to the cart, updating quantity, and calculating total price**

The shopping cart stores selected items temporarily using browser storage and updates dynamically without refreshing the page.

❁ Procedure

1. **Create a project folder named *ecommerce_website***
2. **Create three files: index.html, style.css, script.js**
3. **Design homepage with product listing**
4. **Apply CSS styling for layout and cards**
5. **Implement JavaScript functions:**
    - **Add to cart**
    - **Remove items**
    - **Update quantity**
    - **Calculate total price**
6. **Add checkout section with user input**
7. **Display order confirmation message**

Features Implemented

- **Product listing with name, price, and description**
- **"Add to Cart" functionality for each product**
- **Dynamic shopping cart display using JavaScript**
- **Item quantity management with increase (+) and decrease (−) buttons**
- **Option to remove items from the cart**
- **Automatic calculation of individual item total and overall cart total**
- **Real-time cart update without page refresh**
- **Checkout form to collect user details (Name and Address)**
- **Order confirmation message after placing order**
- **Use of browser localStorage to save cart data temporarily**
- **Clean and user-friendly UI with hover effects**

CODE

HTML

Index.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>E-Commerce Website</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

<header>
    🛒 My E-Commerce Store
</header>

<section class="products">
    <h2>Products</h2>
    <div class="product-container">

        <div class="card">
            <h3>Smartphone</h3>
            <p>₹15000</p>
            <p>Latest Android smartphone</p>
            <button onclick="addToCart('Smartphone',15000)">Add to Cart</button>
        </div>

        <div class="card">
            <h3>Laptop</h3>
            <p>₹50000</p>
            <p>High performance laptop</p>
            <button onclick="addToCart('Laptop',50000)">Add to Cart</button>
        </div>

        <div class="card">
            <h3>Headphones</h3>
            <p>₹2000</p>
            <p>Noise cancelling headphones</p>
            <button onclick="addToCart('Headphones',2000)">Add to Cart</button>
```

```html
        <div class="card">
            <h3>Headphones</h3>
            <p>₹2000</p>
            <p>Noise cancelling headphones</p>
            <button onclick="addToCart('Headphones',2000)">Add to Cart</button>
        </div>

    </div>
</section>

<section class="cart">
    <h2>Shopping Cart</h2>
    <div id="cart-items"></div>
    <h3>Total: ₹<span id="total">0</span></h3>
</section>

<section class="checkout">
    <h2>Checkout</h2>
    <input type="text" id="name" placeholder="Enter Name">
    <input type="text" id="address" placeholder="Enter Address">
    <button onclick="checkout()">Place Order</button>
    <p id="message"></p>
</section>

<script src="script.js"></script>
</body>
</html>
```

Style.css

```css
body {
    font-family: 'Segoe UI', sans-serif;
    margin: 0;
    padding: 0;
    background: #eef2f7;
}

/* Header */
header {
    background: linear-gradient(135deg, #1e3c72, #2a5298);
    color: white;
    padding: 20px;
    text-align: center;
    font-size: 24px;
    font-weight: bold;
}

/* Sections */
.products, .cart, .checkout {
    margin: 30px;
    text-align: center;
}

/* Product Grid */
.product-container {
    display: flex;
    justify-content: center;
    gap: 25px;
    flex-wrap: wrap;
}

/* Cards */
.card {
    background: white;
    padding: 20px;
    border-radius: 15px;
    width: 220px;
```

```
.card:hover {
    transform: translateY(-8px);
}

/* Price */
.card p:nth-child(2) {
    font-size: 18px;
    font-weight: bold;
    color: #2a5298;
}

/* Buttons */
button {
    background: #ff6b6b;
    color: white;
    padding: 10px 15px;
    border: none;
    border-radius: 8px;
    cursor: pointer;
    margin-top: 10px;
    transition: 0.3s;
}

button:hover {
    background: #e63946;
}

/* Cart */
.cart-item {
    background: white;
    margin: 10px auto;
    padding: 12px;
    border-radius: 10px;
    width: 300px;
    box-shadow: 0 4px 10px rgba(0,0,0,0.1);
}
```

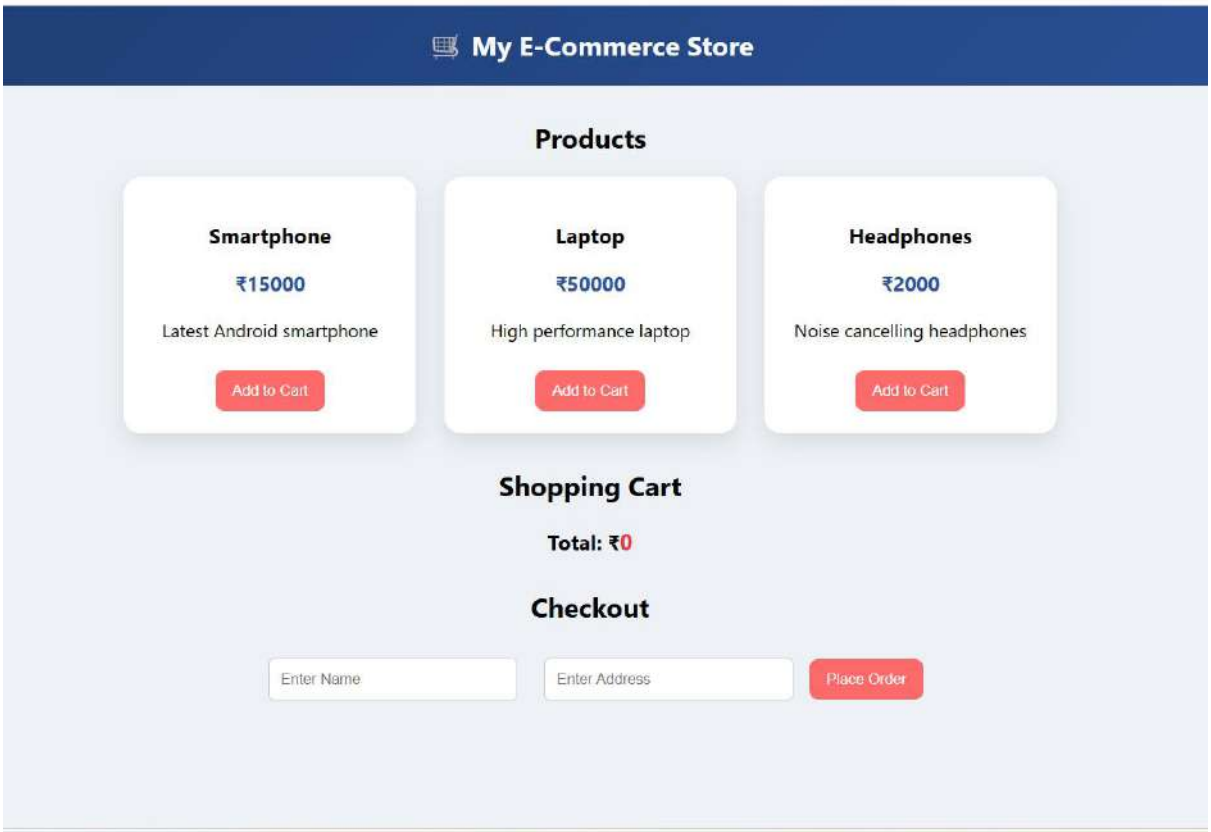
```css
/* Total */
#total {
    color: #e63946;
    font-size: 20px;
    font-weight: bold;
}

/* Checkout */
.checkout input {
    padding: 10px;
    margin: 10px;
    width: 200px;
    border-radius: 5px;
    border: 1px solid #ccc;
}

/* Message */
#message {
    margin-top: 15px;
    font-weight: bold;
    color: green;
}
```

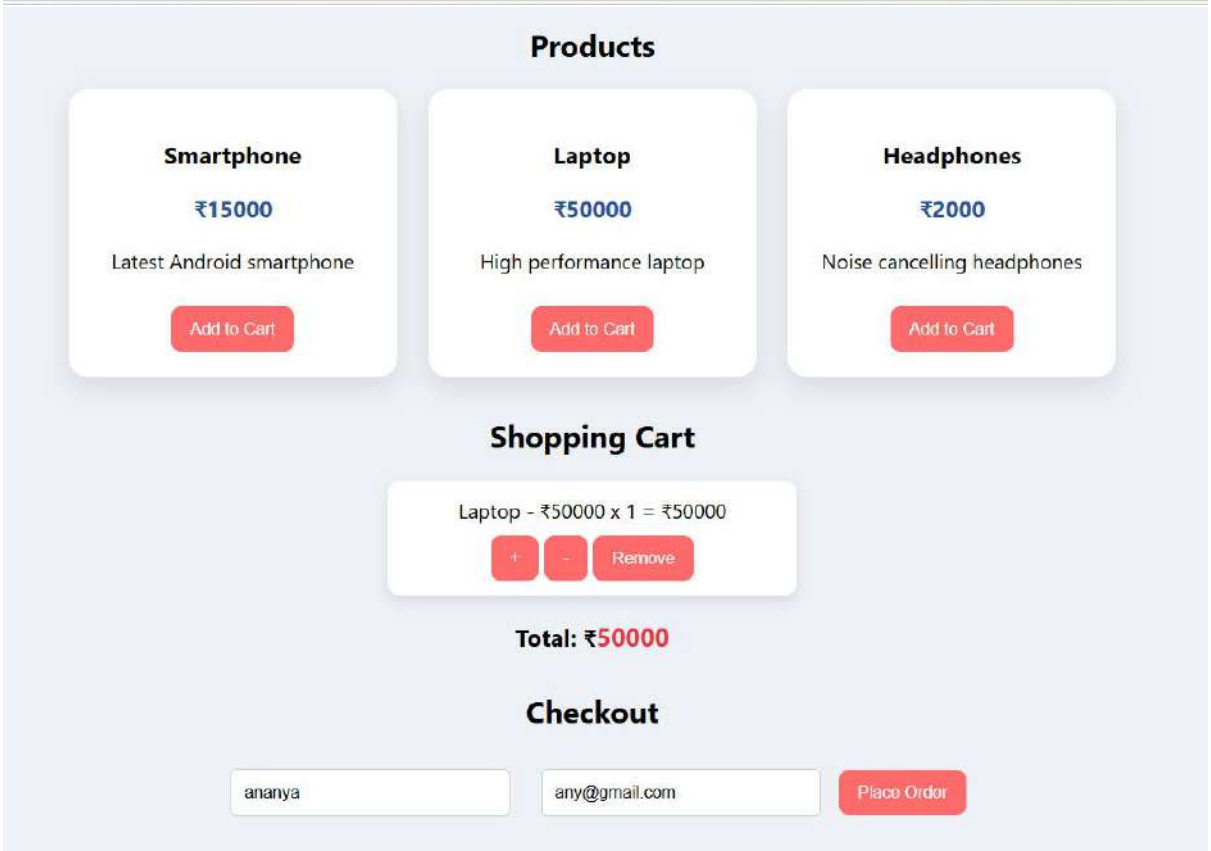
Script.js

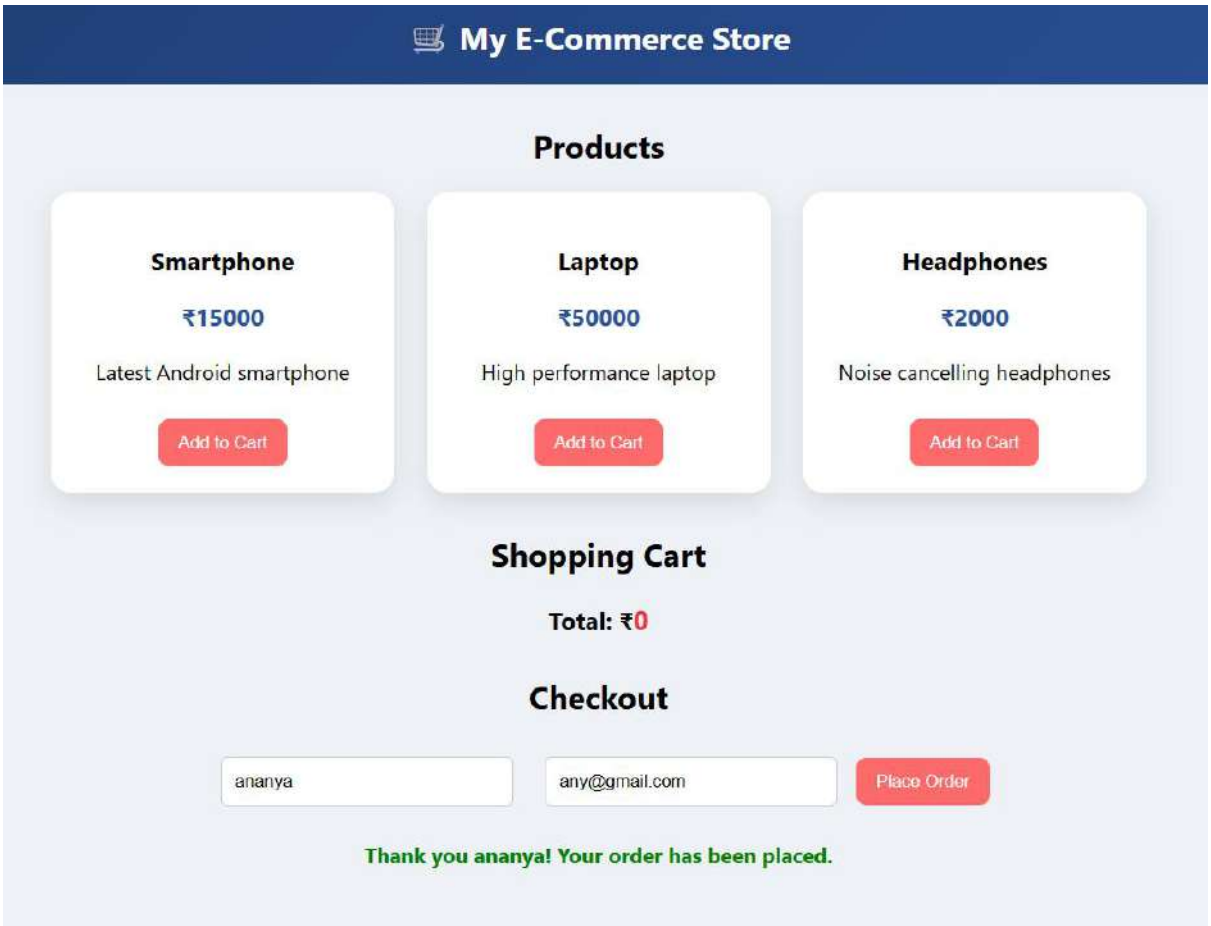
```
let cart = JSON.parse(localStorage.getItem("cart")) || [];

function addToCart(name, price) {
    let item = cart.find(p => p.name === name);

    if (item) {
        item.qty++;
    } else {
        cart.push({ name, price, qty: 1 });
    }

    saveCart();
    displayCart();
}

function displayCart() {
    let cartDiv = document.getElementById("cart-items");
    cartDiv.innerHTML = "";
    let total = 0;

    cart.forEach((item, index) => {
        let div = document.createElement("div");
        div.className = "cart-item";

        let itemTotal = item.price * item.qty;
        total += itemTotal;

        div.innerHTML = `
            ${item.name} - ₹${item.price} x ${item.qty} = ₹${itemTotal}
            <br>
            <button onclick="changeQty(${index},1)">+</button>
            <button onclick="changeQty(${index},-1)">-</button>
            <button onclick="removeItem(${index})">Remove</button>
        `;

        cartDiv.appendChild(div);
    });
```

```
function changeQty(index, change) {

    if (cart[index].qty <= 0) {
        cart.splice(index, 1);
    }

    saveCart();
    displayCart();
}

function removeItem(index) {
    cart.splice(index, 1);
    saveCart();
    displayCart();
}

function saveCart() {
    localStorage.setItem("cart", JSON.stringify(cart));
}

function checkout() {
    let name = document.getElementById("name").value;

    if (cart.length === 0) {
        alert("Cart is empty!");
        return;
    }

    document.getElementById("message").innerText =
        "Thank you " + name + "! Your order has been placed.";

    cart = [];
    saveCart();
    displayCart();
}

// Load cart on refresh
```

OUTPUTS


My E-Commerce Store
Products
Smartphone
₹15000
Latest Android smartphone
Add to Cart
Laptop
₹50000
High performance laptop
Add to Cart
Headphones
₹2000
Noise cancelling headphones
Add to Cart
Shopping Cart
Total: ₹0
Checkout
Enter Name
Enter Address
Place Order


Products
Smartphone
₹15000
Latest Android smartphone
Add to Cart
Laptop
₹50000
High performance laptop
Add to Cart
Headphones
₹2000
Noise cancelling headphones
Add to Cart
Shopping Cart
Laptop - ₹50000 x 1 = ₹50000
+
-
Remove
Total: ₹50000
Checkout
ananya
any@gmail.com
Place Order



☑ Result

The e-commerce website was successfully created with working shopping cart functionality and checkout simulation using HTML, CSS, and JavaScript.